References:

- [CLRS] chapter 35
- Lecture Notes 9, 10
- AAW
- **Sanjoy Dasgupta, Christos Papadimitriou, Umesh Vazirani,** *"Algorithms,"* McGraw-Hill, 2008.
- **Jon Kleinberg and Eva Tardos,** *"Algorithm Design,"* Addison Wesley, 2006.
- **Vijay Vazirani**, "*Approximation Algorithms*," Springer, 2001.
- **Dorith Hochbaum (editor),** *"Approximation Algorithms for NP-Hard Problems,"* PWS Publishing Company, 1997.
- **Michael R. Garey, David S. Johnson,** *"Computers and Intractability: A Guide to the Theory of NP-Completeness,"* W.H. Freeman and Company, 1979.
- **Fedor V. Fomin, Petteri Kaski,** *"Exact Exponential Algorithms: Discovering surprises in the face of intractability,"* Communications of ACM 56(03), pp: 80-89, March 2013.

COMBINATORIAL OPTIMIZATION

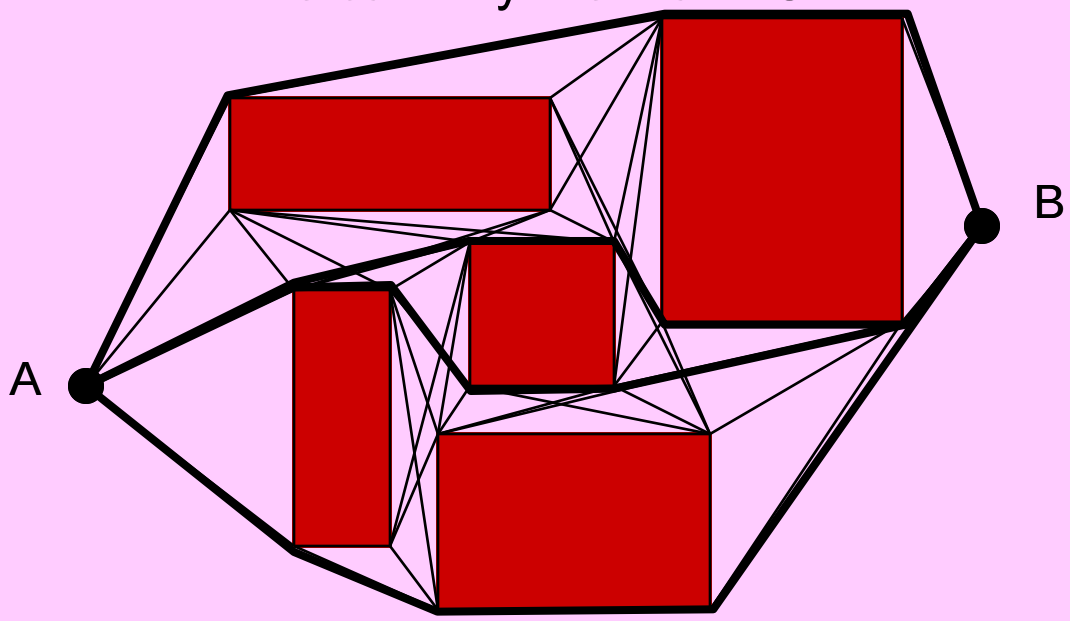
find the best solution out of finitely many possibilities.

Mr. Roboto:
Find the best path from A to B avoiding obstacles

There are infinitely many ways to get from A to B.

With brute-force there are exponentially many paths to try. ☹

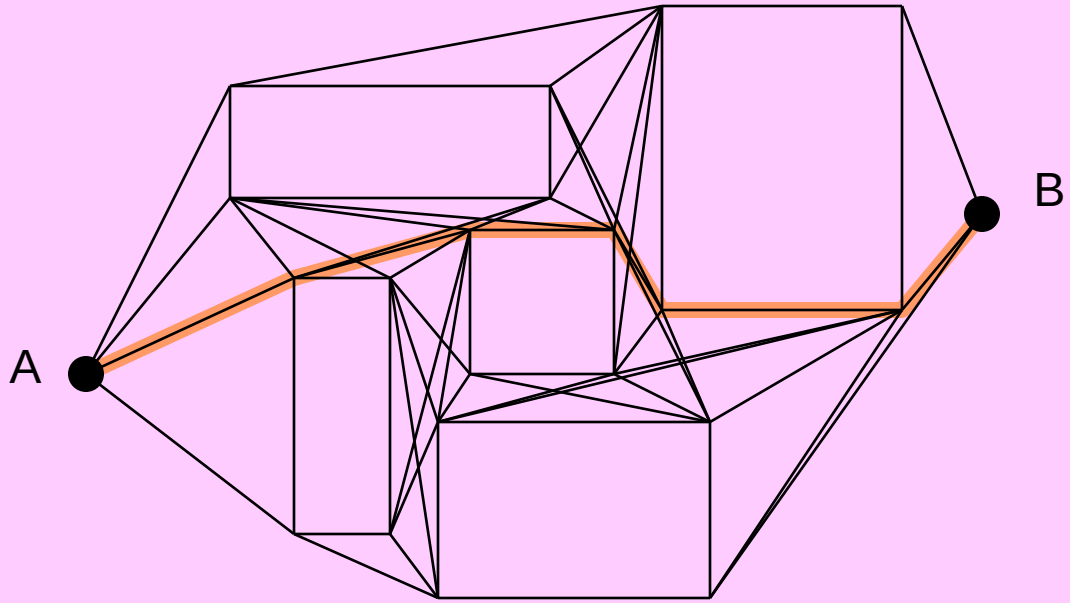
We can't try them all. ☹



There may be a simple and fast (intuitively greedy) strategy. ☺

But you might need to try (nearly) all paths to find the best. ☺

Mr. Roboto:
Find the best path from A to B avoiding obstacles



The Visibility Graph: 4n + 2 vertices
(n = # rectangular obstacles)

Combinatorial Optimization Problem (COP)

INPUT: Instance I to the COP.

Feasible Set: FEAS(I) = the set of all feasible (or valid) solutions for instance I, usually expressed by a set of constraints.

Objective Cost Function:
Instance I includes a description of the objective cost function, Cost[I] that maps each solution S (feasible or not) to a real number or $\pm\infty$.

Goal: Optimize (i.e., minimize or maximize) the objective cost function.

Optimum Set:
$$OPT(I) = \{ Sol \in FEAS(I) \mid Cost[I](Sol) \leq Cost[I](Sol'), \ \forall Sol' \in FEAS(I) \}$$
the set of all minimum cost feasible solutions for instance I

Combinatorial: Means the problem structure implies that only a finite number of solutions need to be examined to find the optimum.

OUTPUT: A solution $Sol \in OPT(I)$, or report that $FEAS(I) = \varnothing$.

Polynomial vs Exponential time

Assume: Computer speed 10^6 IPS and input size n = 10^6

Time complexity	Running time
n	1 sec.
n log n	20 sec.
n^2	12 days
2^n	40 quadrillion (10^{15}) years

COP Examples

- **"Easy"** (polynomial-time solvable):
  - Shortest (simple) Path [AAW, Dijkstra, Bellman-Ford, Floyd-Warshall …]
  - Minimum Spanning Tree [AAW, Prim, Kruskal, Cheriton-Tarjan, …]
  - Graph Matching [Jack Edmonds, …]

- **"NP-Hard"** (no known polynomial-time solution):
  - Longest (simple) Path
  - Traveling Salesman
  - Vertex Cover
  - Set Cover
  - K-Cluster
  - 0/1 Knapsack

Example: Primality Testing

Given integer N ≥ 2, is N a prime number?

```
for  i ← 2 .. N -1  do  if  N mod i = 0  then return  NO
return YES
```

Input bit-size: $b = \log N$.

Running time: $O(N) = O(2^b)$.

This is an **exponential** time algorithm.

There is a **polynomial** time primality testing algorithm:

Manindra Agrawal, Neeraj Kayal, and Nitin Saxena,
"PRIMES is in P," Annals of Mathematics, 160 (2004), 781–793.

Punch line: be careful when time complexity also depends on values of input **numeric** data (as opposed to combinatorial measures such as the number of vertices in the graph, or the number of items in the input array, etc.).

Approximation Algorithm for NP-Hard **COP**

- Algorithm A: polynomial time on any input (bit-) size n.
- S_A feasible solution obtained by algorithm A
- S_{OPT} optimum solution.
- $C(S_A) > 0$ cost of solution obtained by algorithm A
- $C(S_{OPT}) > 0$ cost of optimum solution
- $\rho = \rho(n) > 1$ (worst-case) approximation ratio

- **A is a ρ-approximation algorithm if for all instances:**

$$1/\rho \leq C(S_A) / C(S_{OPT}) \leq \rho$$

for Maximization

for Minimization

Design Methods

- Greedy Methods [e.g., Weighted Set Cover, Clustering]
- Cost Scaling or Relaxation [e.g., 0/1 Knapsack, LN9]
- Constraint Relaxation [e.g., Weighted Vertex Cover]
- Combination of both [e.g., Lagrangian relaxation]
- LP-relaxation of Integer Linear Program (ILP):
  - LP-rounding method
  - LP primal-dual method [e.g., the Pricing Method]
- Trimmed Dynamic Programming [e.g., Arora: Euclidean-TSP, LN10]

- Local Search Heuristics [e.g., 2-exchange in TSP]
  - Tabu Search
  - Genetic Heuristics
  - Simulated Annealing
  - •••

Analysis Method

- Establish cost lower/upper bounds LB and UB such that:
  1) $LB \leq C(S_A) \leq UB$
  2) $LB \leq C(S_{OPT}) \leq UB$
  3) $UB \leq \rho\, LB$

- **Minimization:** $LB \leq C(S_{OPT}) \leq C(S_A) = UB \leq \rho\, LB \leq \rho\, C(S_{OPT})$

  $\therefore \quad C(S_{OPT}) \leq C(S_A) \leq \rho\, C(S_{OPT})$

  Also: $C(S_A) \leq (1+\varepsilon)\, C(S_{OPT})$ [$\varepsilon>0$ relative error. $\rho = 1+\varepsilon$]

- **Maximization:** $LB = C(S_A) \leq C(S_{OPT}) \leq UB \leq \rho\, LB = \rho\, C(S_A)$

  $\therefore\ C(S_A) \leq C(S_{OPT}) \leq \rho\, C(S_A)$

  Also: $C(S_A) \geq (1-\varepsilon)\, C(S_{OPT})$ [$1>\varepsilon>0$ relative error. $1/\rho = 1-\varepsilon$]

Classes of Approximation Methods

- ρ-approximation algorithm A:
  - (1) runs in time polynomial in the input (bit-) size, and
  - (2) $1/\rho \leq C(S_A) / C(S_{OPT}) \leq \rho$

- PTAS: Polynomial-Time Approximation Scheme:
  - Additional input parameter ε (as desired relative error bound)
  - (1) it finds a solution with relative error at most ε, and
  - (2) for any fixed ε, it runs in time polynomial in the input (bit-) size.
    [For example, $O(n^{1/\varepsilon})$.]

- FPTAS: Fully Polynomial-Time Approximation Scheme:
  - (1) is a PTAS, and
  - (2) running time is polynomial in both the input (bit-) size and in $1/\varepsilon$.
    [For example, $O(1/\varepsilon\, n^2)$.]

NP-hard COPs studied here

- Weighted Vertex Cover Problem
- Weighted Set Cover Problem
- Traveling Salesman Problem
- K-Cluster Problem
- 0/1 Knapsack problem

Weighted Vertex Cover Problem

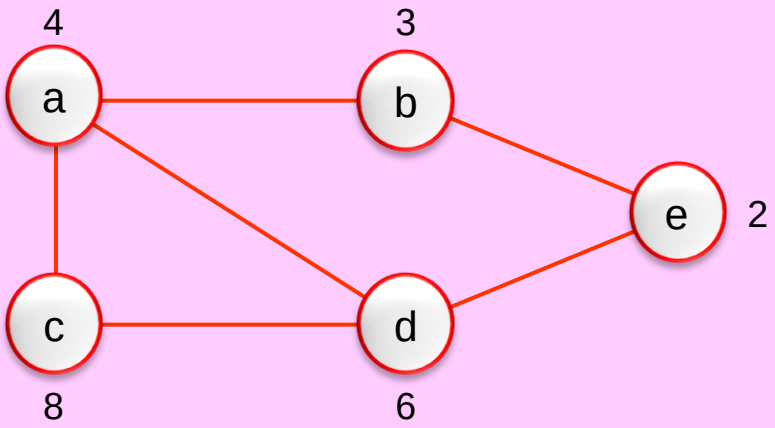
Weighted Vertex Cover Problem (WVCP)

Input: an undirected graph G(V,E) with vertex weights w(V). w(v)>0 is weight of vertex $v \in V$.

Output: a vertex cover C: a subset $C \subseteq V$ that "hits" (or covers) all edges, i.e., $\forall (u,v) \in E$, $u \in C$ or $v \in C$ (or both).

Goal: minimize weight of vertex cover C: $W(C) = \sum_{v \in C} w(v)$.

[CLRS35] considers the un-weighted case only, i.e., w(v) = 1 $\forall v \in V$.



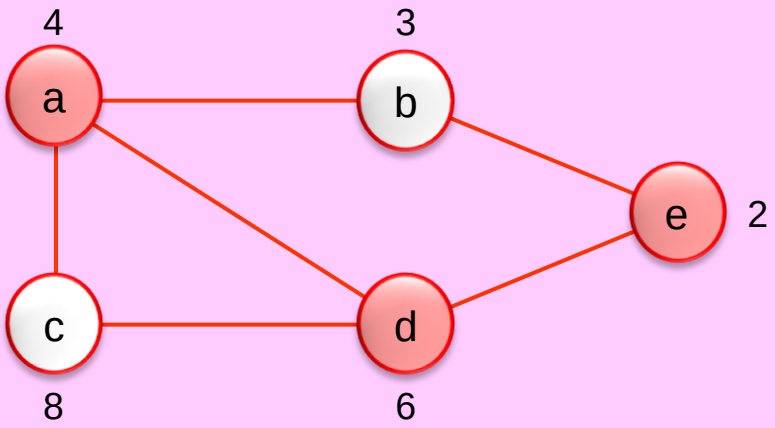
Weighted Vertex Cover Problem (WVCP)

Input: an undirected graph G(V,E) with vertex weights w(V). w(v)>0 is weight of vertex $v \in V$.

Output: a vertex cover C: a subset $C \subseteq V$ that "hits" (or covers) all edges, i.e., $\forall (u,v) \in E$, $u \in C$ or $v \in C$ (or both).

Goal: minimize weight of vertex cover C: $W(C) = \sum_{v \in C} w(v)$.

[CLRS35] considers the un-weighted case only, i.e., w(v) = 1 $\forall v \in V$.



C_{OPT} = { a, d, e}

$W(C_{OPT})$ = 4 + 6 + 2 = 12

WVCP as an ILP

- 0/1 variables on vertices:

$$\forall v \in V: \quad x(v) = \begin{cases} 1 & \text{if } v \in C \\ 0 & \text{if } v \notin C \end{cases}$$

- **(P1)** WVCP as an ILP:

$$\text{minimize} \quad \sum_{v \in V} w(v)\, x(v)$$

subject to:

$$(1) \quad x(u) + x(v) \geq 1 \qquad \forall (u,v) \in E$$

$$(2) \quad x(v) \in \{0,1\} \qquad \forall v \in V$$

- **(P2)** LP Relaxation:

$$\text{minimize} \quad \sum_{v \in V} w(v)\, x(v)$$

subject to:

$$(1) \quad x(u) + x(v) \geq 1 \qquad \forall (u,v) \in E$$

$$(2') \quad x(v) \geq 0 \qquad \forall v \in V$$

WVCP LB & UB

- **(P2)** LP Relaxation:

$$\text{minimize} \quad \sum_{v \in V} w(v)\, x(v)$$

$$\text{subject to:}$$

$$(1) \quad x(u) + x(v) \geq 1 \qquad \forall (u, v) \in E$$

$$(2') \quad x(v) \geq 0 \qquad \forall v \in V$$

- **(P3)** Dual LP:

$$\text{maximize} \quad \sum_{(u,v) \in E} p(u, v)$$

$$\text{subject to:}$$

$$(3) \quad \sum_{u:\,(v,u) \in E} p(u, v) \leq w(v) \qquad \forall v \in V$$

$$(4) \quad p(u, v) \geq 0 \qquad \forall (u, v) \in E$$

To learn more about LP Duality see my CSE6118 or CSE6114 Lecture Slide on LP .

- OPTcost (P1) ≥ OPTcost (P2) = OPTcost (P3) ≥ feasible cost(P3)

min relaxation **LP Duality** **max problem**

- **LB:** cost of any feasible solution to (P3)
- **UB:** feasible VC by the pricing method (LP primal-dual)

Approx WVCP by Pricing Method

Define dual (real) price variables p(u,v) for each (u,v) $\in$ E

Price Invariant (PI): [maintain (P3) feasibility]:

$$(3) \quad \sum_{u:\,(v,u)\in E} p(u,v) \le w(v) \qquad \forall v \in V$$

$$(4) \quad p(u,v) \ge 0 \qquad \forall (u,v) \in E$$

Interpretation: a vertex (server) v$\in$C covers its incident edges (clients). The weight (cost) of v is distributed as charged price among these clients, without over-charging.

We say vertex v is **tight** if its inequality (3) is met as equality, i.e.,

$$v \text{ is tight if } : \quad \sum_{\substack{u:\\ (u,v)\in E}} p(u,v) = w(v).$$

We say (the price of) an edge (u,v) is **final** if u or v is tight.

Approx WVCP by Pricing Method

ALGORITHM Approximate-Vertex-Cover (G(V,E), w(V))

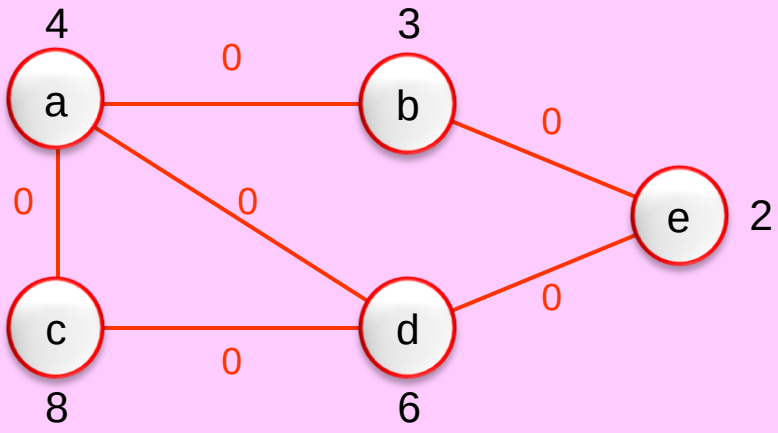
for each edge (u,v)∈E **do** p(u,v) ← 0

for each edge (u,v)∈E **do** finalize (u,v), i.e.,
increase p(u,v) until u or v becomes tight

C ← { v ∈ V | v is tight }

return C

end



Finalize edge:

Approx WVCP by Pricing Method

ALGORITHM Approximate-Vertex-Cover (G(V,E), w(V))

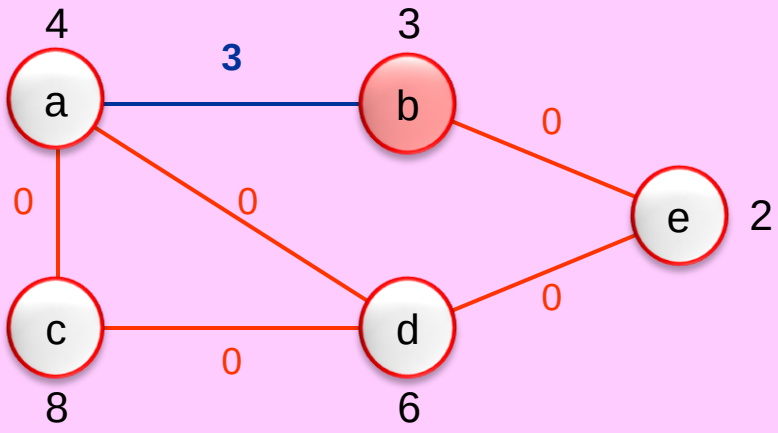
for each edge (u,v)∈E **do** p(u,v) ← 0

for each edge (u,v)∈E **do** finalize (u,v), i.e.,
increase p(u,v) until u or v becomes tight

C ← { v ∈ V | v is tight }

return C

end



Finalize edge:
(a,b) : b becomes tight

Approx WVCP by Pricing Method

ALGORITHM Approximate-Vertex-Cover (G(V,E), w(V))

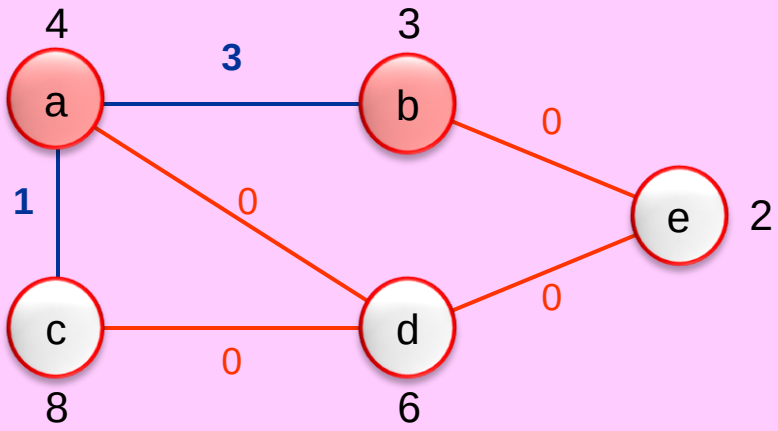
for each edge (u,v)∈E **do** p(u,v) ← 0

for each edge (u,v)∈E **do** finalize (u,v), i.e.,
increase p(u,v) until u or v becomes tight

C ← { v ∈ V | v is tight }

return C

end



Finalize edge:
(a,b) : b becomes tight
(a,c) : a becomes tight

Approx WVCP by Pricing Method

ALGORITHM Approximate-Vertex-Cover (G(V,E), w(V))

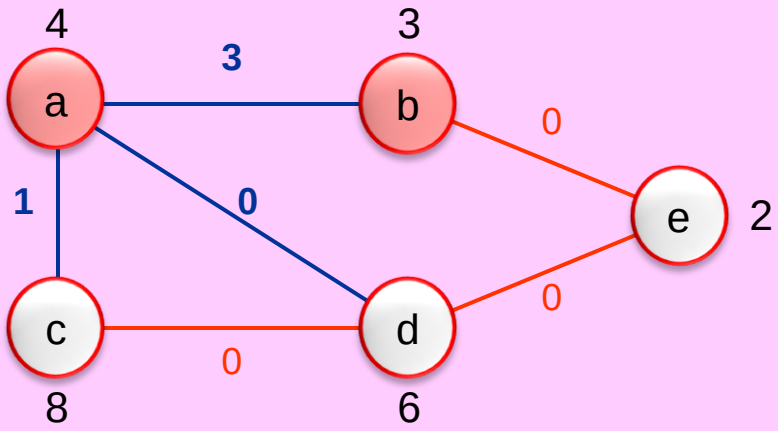
for each edge (u,v)∈E **do** p(u,v) ← 0

for each edge (u,v)∈E **do** finalize (u,v), i.e.,
increase p(u,v) until u or v becomes tight

C ← { v ∈ V | v is tight }

return C

end



Finalize edge:
(a,b) : b becomes tight
(a,c) : a becomes tight
(a,d) : no change

Approx WVCP by Pricing Method

ALGORITHM Approximate-Vertex-Cover (G(V,E), w(V))

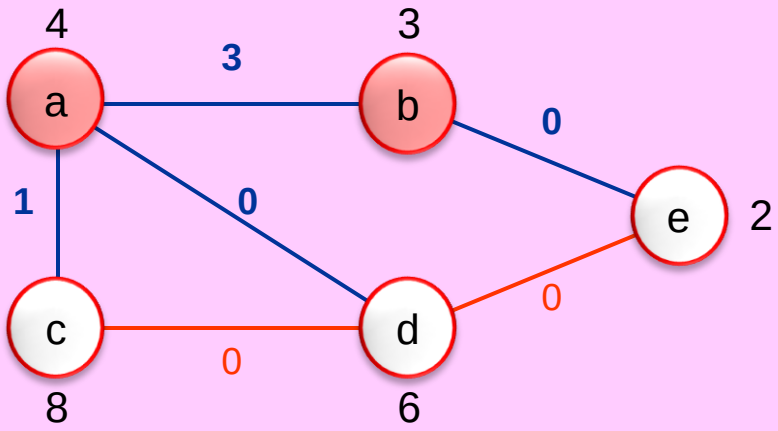
for each edge (u,v)∈E **do** p(u,v) ← 0

for each edge (u,v)∈E **do** finalize (u,v), i.e.,
increase p(u,v) until u or v becomes tight

C ← { v ∈ V | v is tight }

return C

end



Finalize edge:
(a,b) : b becomes tight
(a,c) : a becomes tight
(a,d) : no change
(b,e) : no change

Approx WVCP by Pricing Method

ALGORITHM Approximate-Vertex-Cover (G(V,E), w(V))

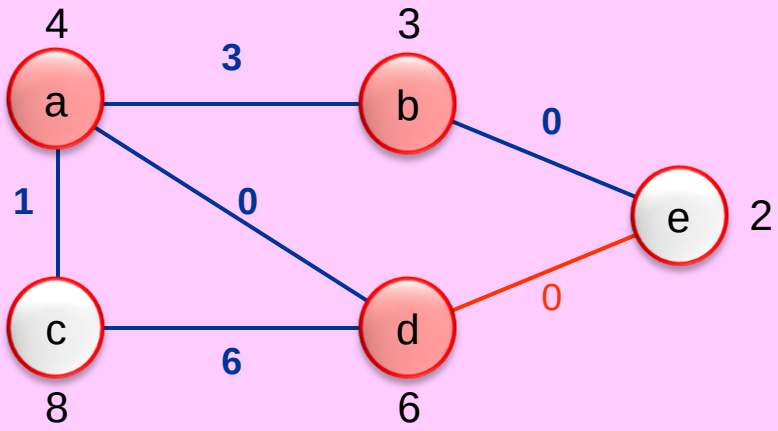
for each edge (u,v)∈E **do** p(u,v) ← 0

for each edge (u,v)∈E **do** finalize (u,v), i.e.,
increase p(u,v) until u or v becomes tight

C ← { v ∈ V | v is tight }

return C

end



Finalize edge:
(a,b) : b becomes tight
(a,c) : a becomes tight
(a,d) : no change
(b,e) : no change
(c,d) : d becomes tight

Approx WVCP by Pricing Method

ALGORITHM Approximate-Vertex-Cover (G(V,E), w(V))

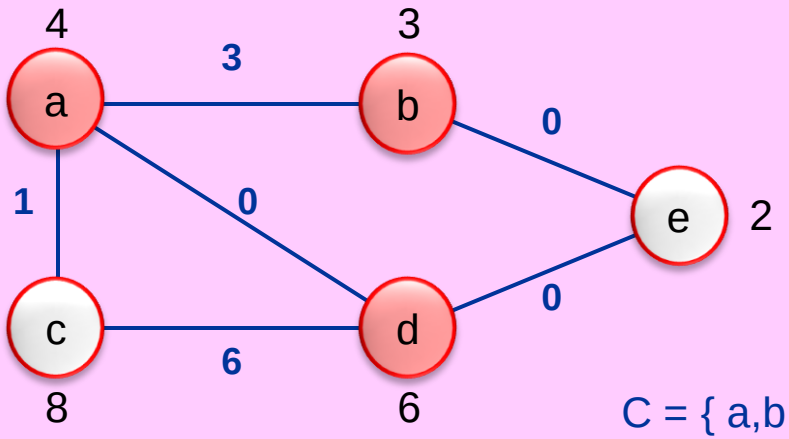
for each edge (u,v)∈E **do** p(u,v) ← 0

for each edge (u,v)∈E **do** finalize (u,v), i.e.,
increase p(u,v) until u or v becomes tight

C ← { v ∈ V | v is tight }

return C

end



Finalize edge:
(a,b) : b becomes tight
(a,c) : a becomes tight
(a,d) : no change
(b,e) : no change
(c,d) : d becomes tight
(d,e) : no change

C = { a,b,d} , W(C) = 4 + 3 + 6 = 13
C_{OPT} = {a,d,e}, $W(C_{OPT})$ = 4+6+2 = 12.

This is a 2-approximation algorithm

THEOREM: The algorithm has the following properties:

(1) Correctness: outputs a feasible vertex cover C

(2) Running Time: polynomial (in fact linear time)

(3) Approx Bound: $W(C) \leq 2\ W(C_{OPT})$ (and $\rho = 2$ is tight)

Proof of (1): By the end of the algorithm all edges are finalized, i.e., have at least one tight incident vertex. That tight vertex is in C. Hence, C covers all edges of G.

Proof of (2): Obvious. For each vertex maintain its PI non-negative slack.

This is a 2-approximation algorithm

THEOREM: The algorithm has the following properties:

(1) Correctness: outputs a feasible vertex cover C

(2) Running Time: polynomial (in fact linear time)

(3) Approx Bound: $W(C) \leq 2\ W(C_{OPT})$ (and $\rho = 2$ is tight)

Proof of (3): Since it's a minimization problem, we know $W(C_{OPT}) \leq W(C) = UB$.

$$W(C_{OPT}) = \sum_{v \in C_{OPT}} w(v) \underset{PI}{\geq} \sum_{v \in C_{OPT}} \sum_{\substack{u: \\ (u,v) \in E}} p(u,v) \underset{\substack{C_{OPT} \\ \text{is a VC}}}{\geq} \sum_{(u,v) \in E} p(u,v) = LB.$$

$$UB = W(C) = \sum_{v \in C} w(v) \underset{tight}{=} \sum_{v \in C} \sum_{\substack{u: \\ (u,v) \in E}} p(u,v) \leq 2 \sum_{(u,v) \in E} p(u,v) = 2\ LB.$$

Therefore, $W(C) \leq 2\ W(C_{OPT})$.

[$\rho = 2$ is tight: Consider a graph with one edge and 2 equal-weight vertices.]

Weighted Set Cover Problem

Weighted Set Cover Problem (WSCP)

Input: a set $X = \{x_1, x_2, \ldots, x_n\}$ of n elements,
a family $F = \{S_1, S_2, , \ldots, S_m\}$ of m subsets of X, and
w(F): weights $w(S)>0$, for each $S \in F$.

Pre-condition: F covers X, i.e., $X = \cup_{S \in F} S = S_1 \cup S_2 \cup \ldots \cup S_m$.

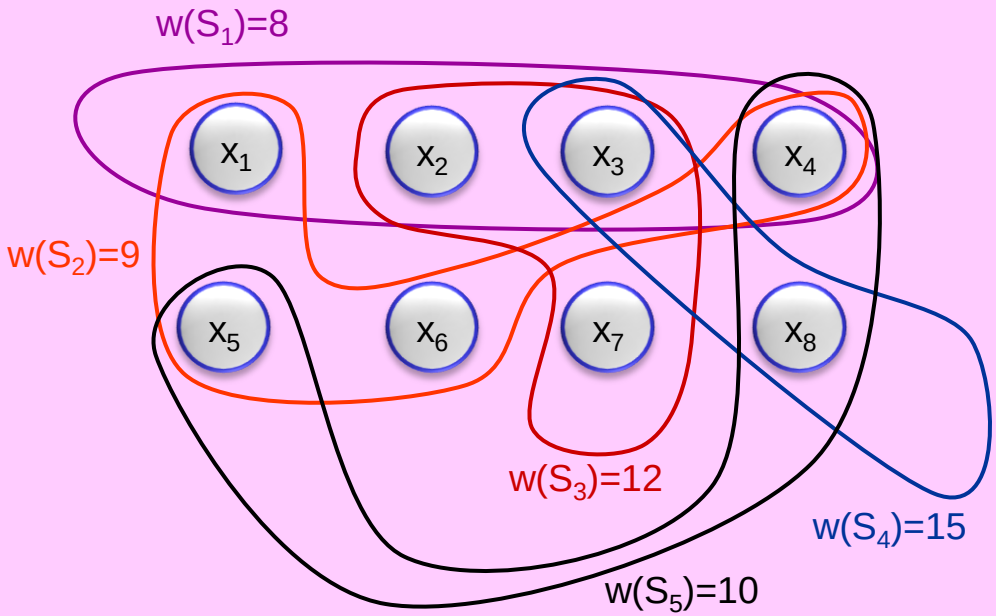
Output: a subset $C \subseteq F$ that covers X, i.e., $X = \cup_{S \in C} S$.

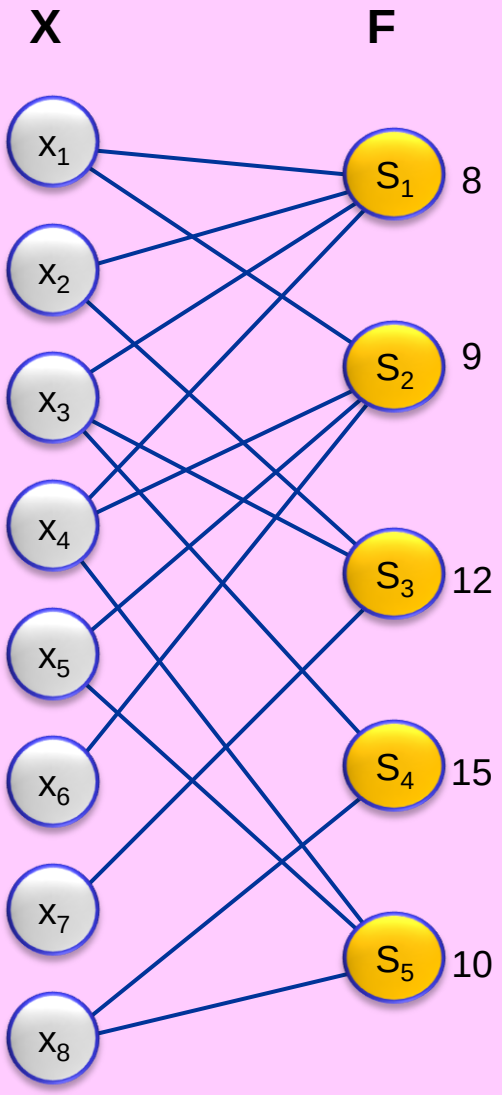
Goal: minimize weight of set cover C: $W(C) = \sum_{S \in C} w(S)$.

[CLRS35] considers the un-weighted case only, i.e., $w(S) = 1 \ \forall S \in F$.

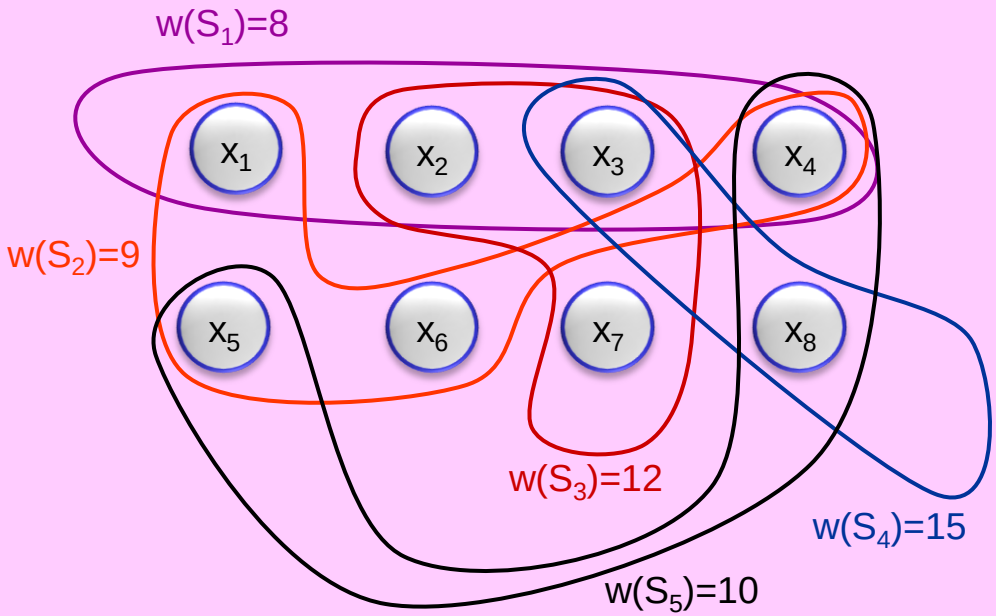
Set Cover (SC) generalizes Vertex Cover (VC):
In VC, elements (clients) are edges, and sets (servers) correspond to vertices.
The set corresponding to a vertex v is the set of edges incident to v.

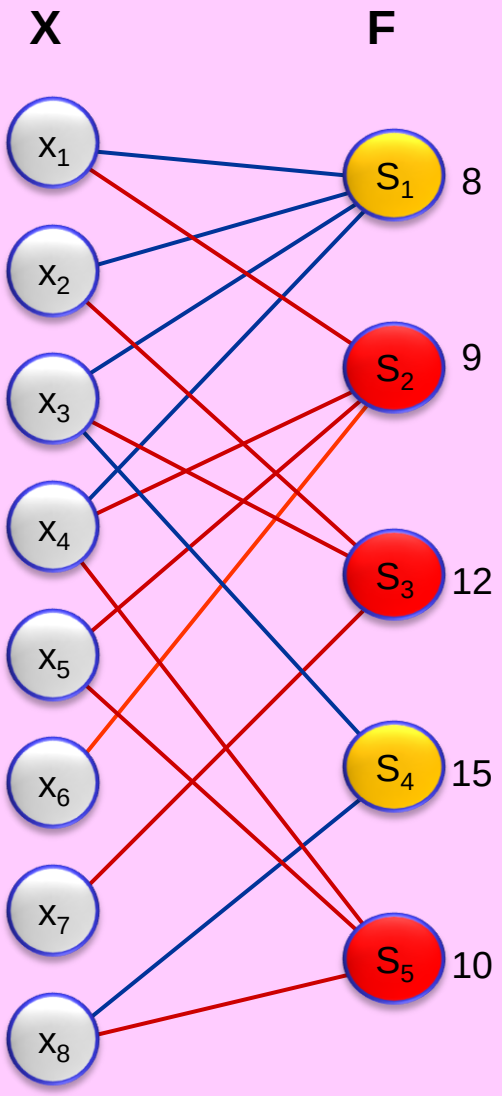
Example


w(S1)=8
w(S2)=9
w(S3)=12
w(S4)=15
w(S5)=10
x1
x2
x3
x4
x5
x6
x7
x8


X
F
x1
x2
x3
x4
x5
x6
x7
x8
S1
S2
S3
S4
S5
8
9
12
15
10

Example





$C_{OPT} = \{ S_2, S_3, S_5 \}$

$W(C_{OPT}) = 9 + 12 + 10 = 31$

WSCP as an ILP

- WSCP ILP:

$$\begin{aligned} &\textit{minimize} && \textstyle\sum_{S\in F} && w(S)\, z(S) && \\ &\textit{subject to:} && \textstyle\sum_{\substack{S\in F:\\ x\in S}} && z(S) \geq 1 && (\forall x \in X) \\ & && && z(S) \in \{0,1\} && (\forall S \in F) \end{aligned}$$

- LP Relaxation:

$$\begin{aligned} &\textit{minimize} && \textstyle\sum_{S\in F} && w(S)\, z(S) && \\ &\textit{subject to:} && \textstyle\sum_{\substack{S\in F:\\ x\in S}} && z(S) \geq 1 && (\forall x \in X) \\ & && && z(S) \geq 0 && (\forall S \in F) \end{aligned}$$

- Dual LP:

$$\begin{aligned} &\textit{maximize} && \textstyle\sum_{x\in X} && p(x) && \\ &\textit{subject to:} && \textstyle\sum_{x\in S} && p(x) \leq w(S) && (\forall S \in F) \\ & && && p(x) \geq 0 && (\forall x \in X) \end{aligned}$$

Provides Lower Bound

Approx WSCP by Pricing Method

ALGORITHM Greedy-Set-Cover (X, F, w(F))

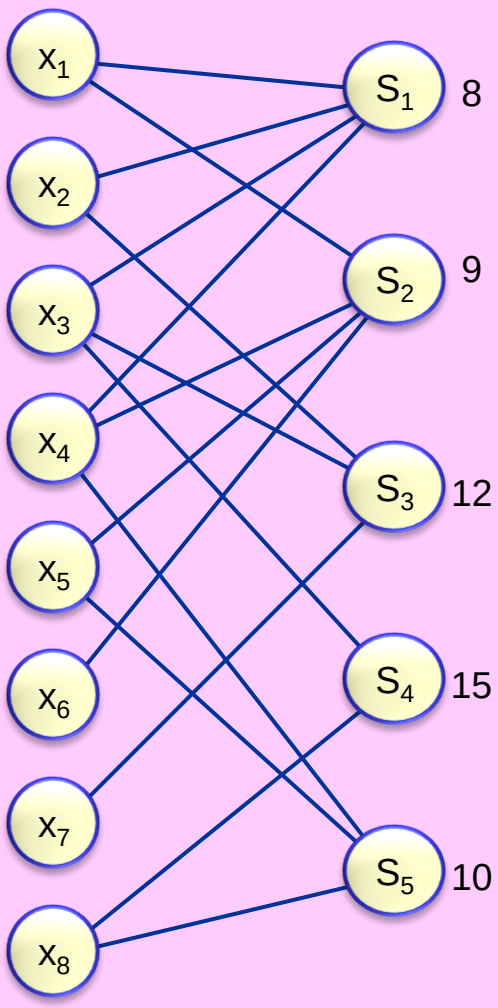
1. $U \leftarrow X$ (* uncovered elements *)
2. $C \leftarrow \varnothing$ (* set cover *)
3. **while** $U \neq \varnothing$ **do**
4. select $S \in F$ that minimizes price $p = w(S) / |S \cap U|$
5. $U \leftarrow U - S$
6. $C \leftarrow C \cup \{S\}$
7. **return** C

end

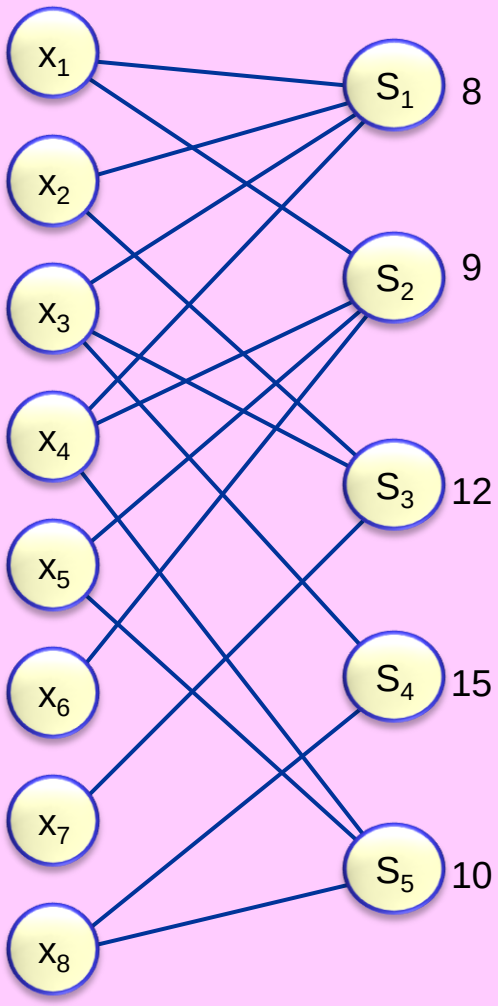
Definition [for the sake of analysis]:
Price p(x) charged to an element $x \in X$ is the price (at line 4) at the earliest iteration that x gets covered (i.e., removed from U at line 5).
[Note: p(x) is charged to x only once and at the first time x gets covered.]

Example run of the algorithm

Iteration : p = w(S) / |S∩U|



Example run of the algorithm



Iteration : p = w(S) / |S∩U|

8/4

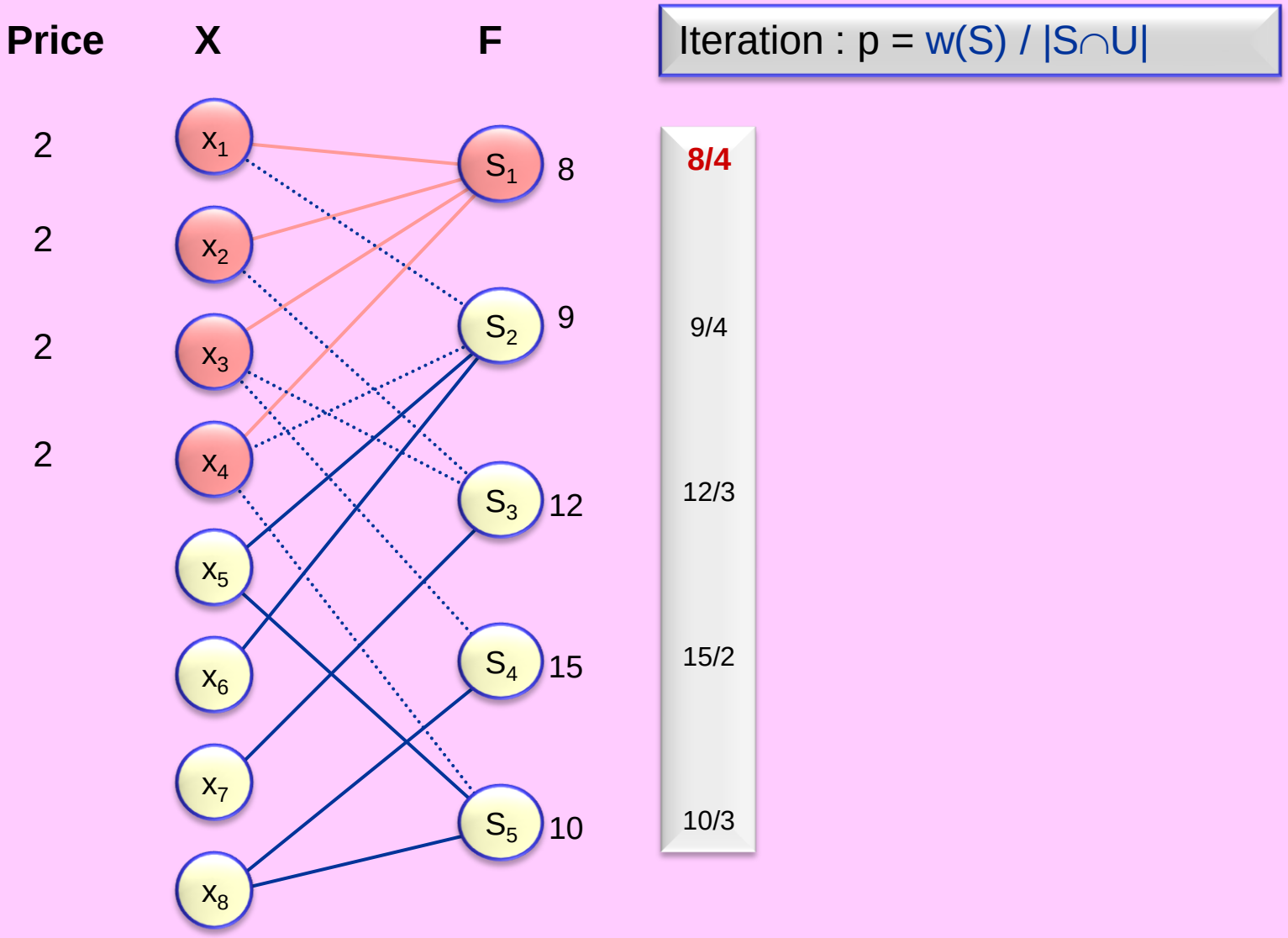
9/4

12/3

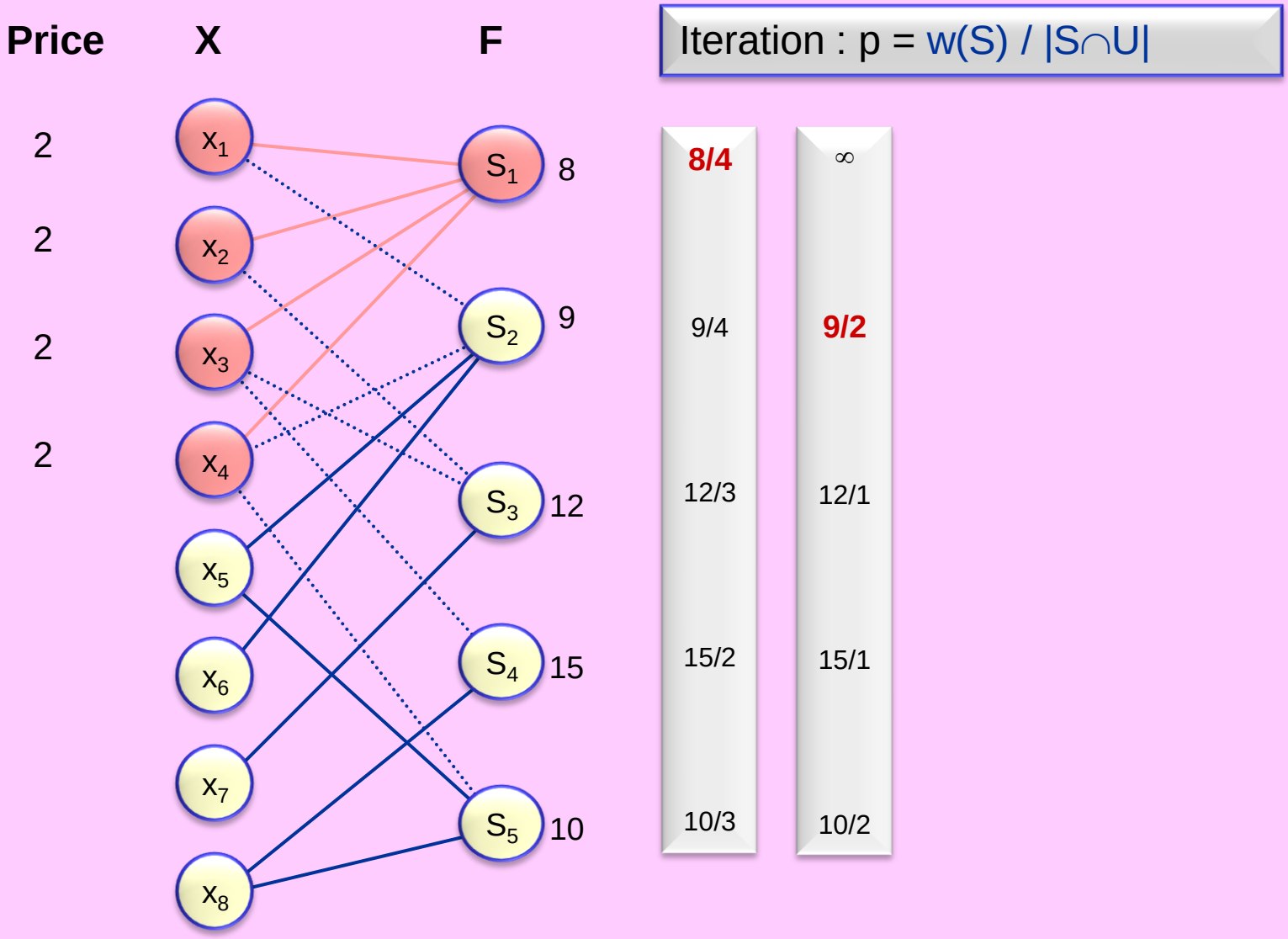
15/2

10/3

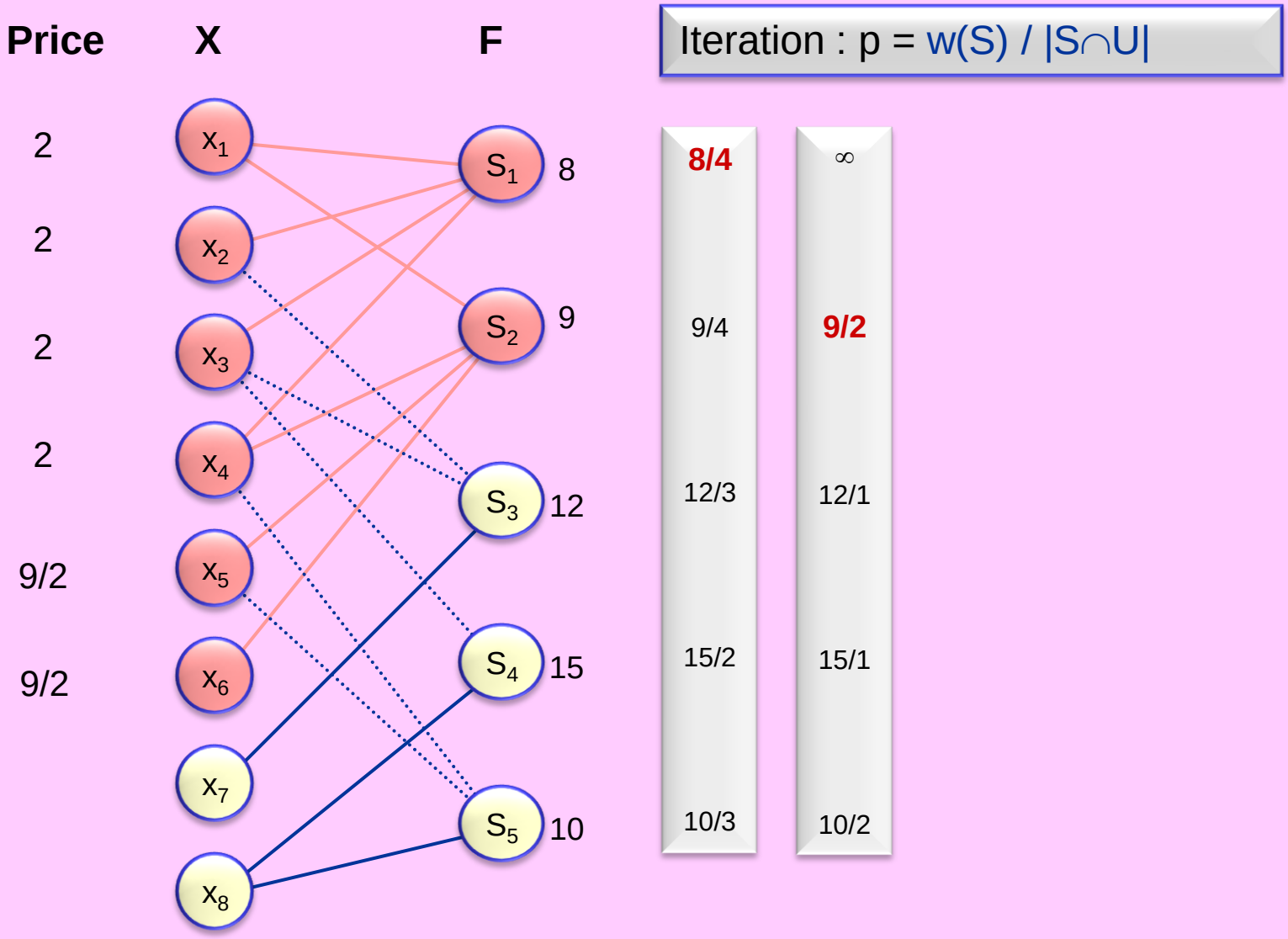
Example run of the algorithm



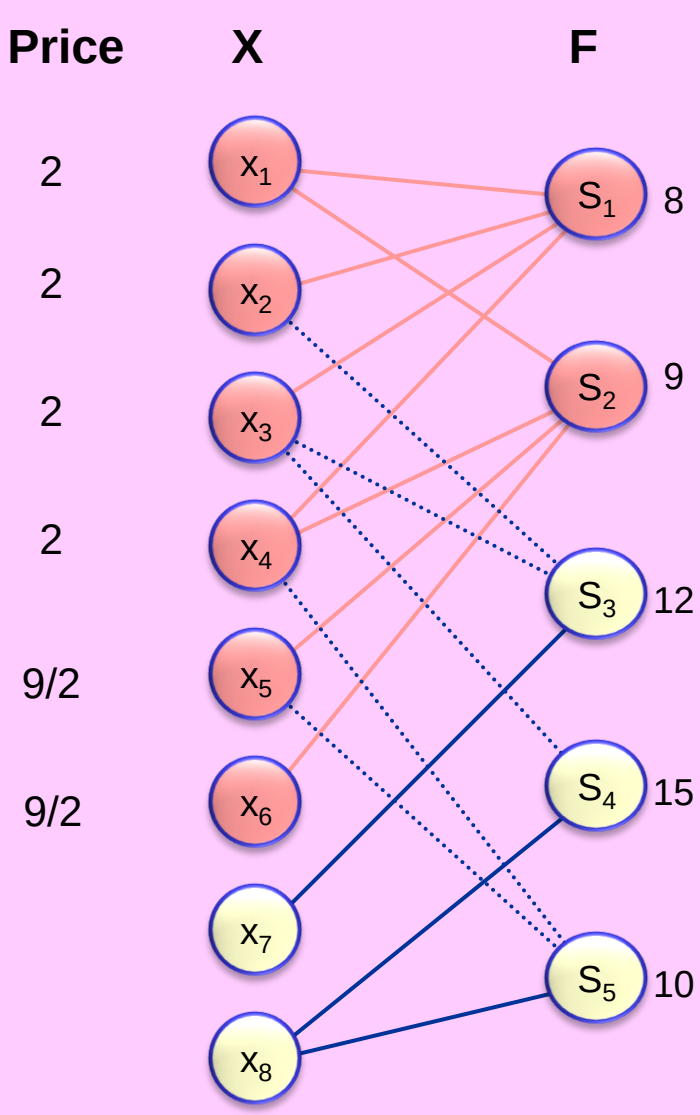
Example run of the algorithm



Example run of the algorithm



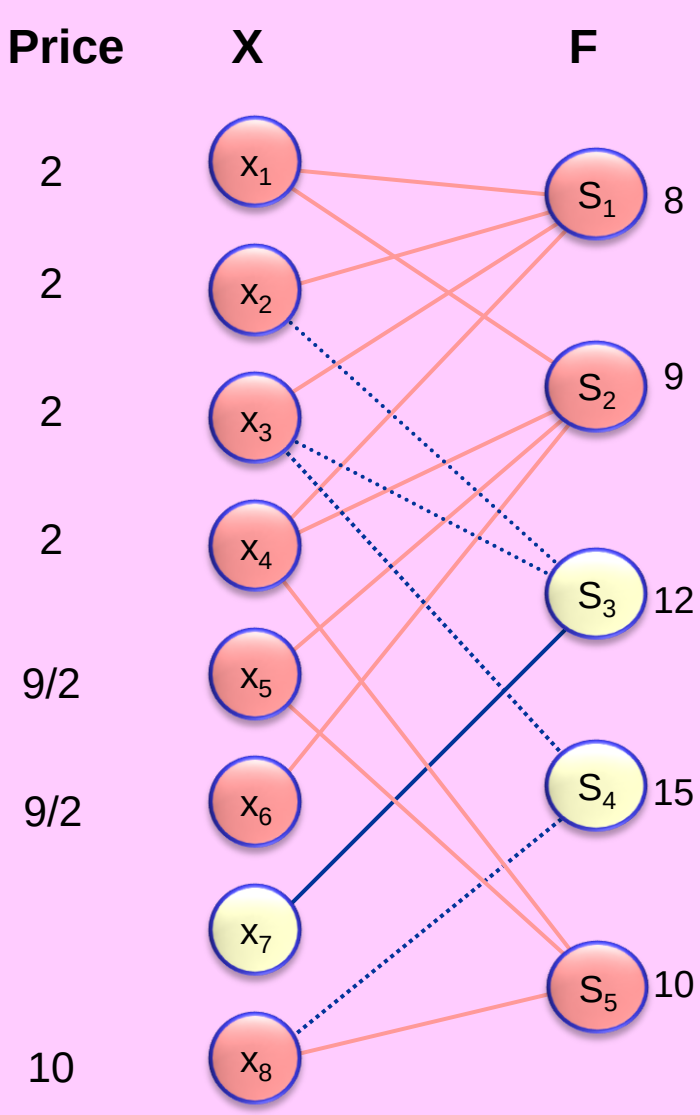
Example run of the algorithm



Iteration : p = w(S) / |S∩U|

8/4	∞	∞
9/4	**9/2**	∞
12/3	12/1	12/1
15/2	15/1	15/1
10/3	10/2	**10/1**

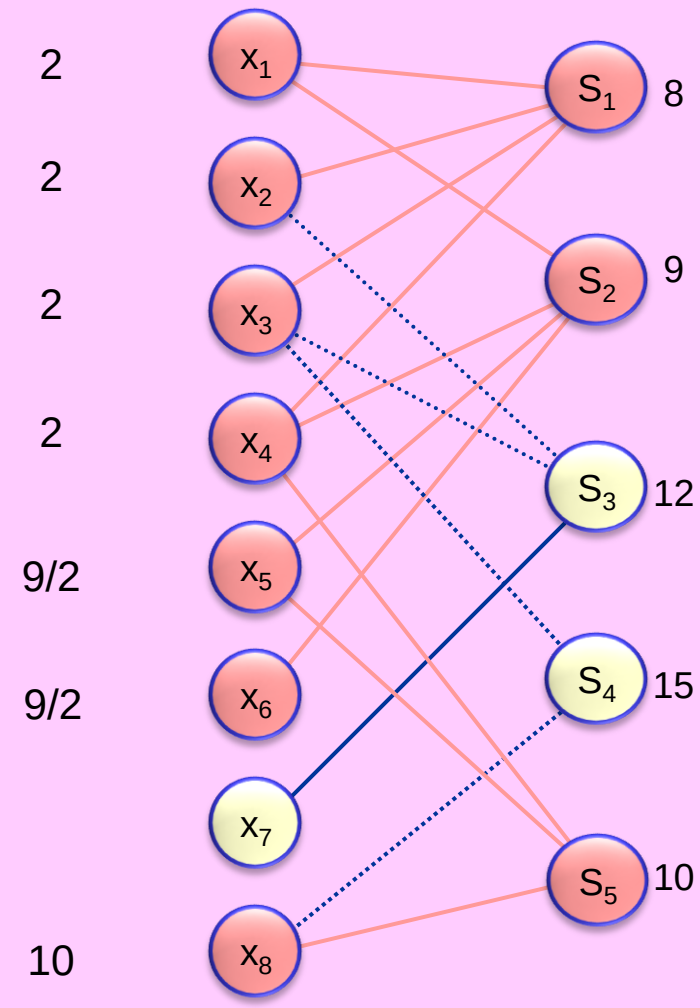
Example run of the algorithm



Iteration : p = w(S) / |S∩U|

8/4	∞	∞
9/4	**9/2**	∞
12/3	12/1	12/1
15/2	15/1	15/1
10/3	10/2	**10/1**

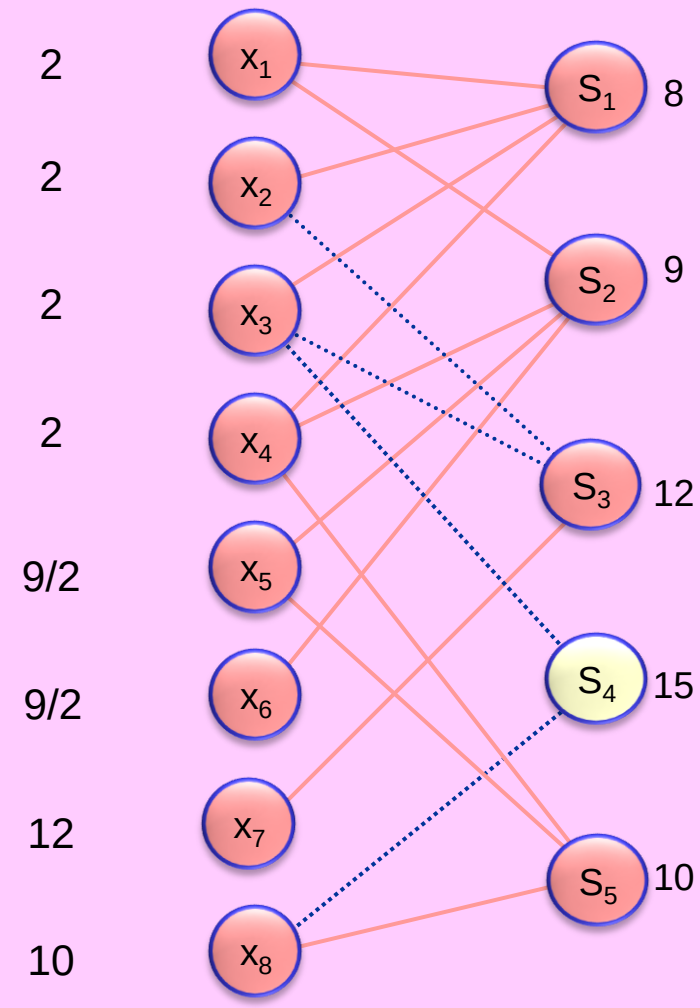
Example run of the algorithm



Iteration : p = w(S) / |S∩U|

8/4	∞	∞	∞
9/4	**9/2**	∞	∞
12/3	12/1	12/1	**12/1**
15/2	15/1	15/1	∞
10/3	10/2	**10/1**	∞

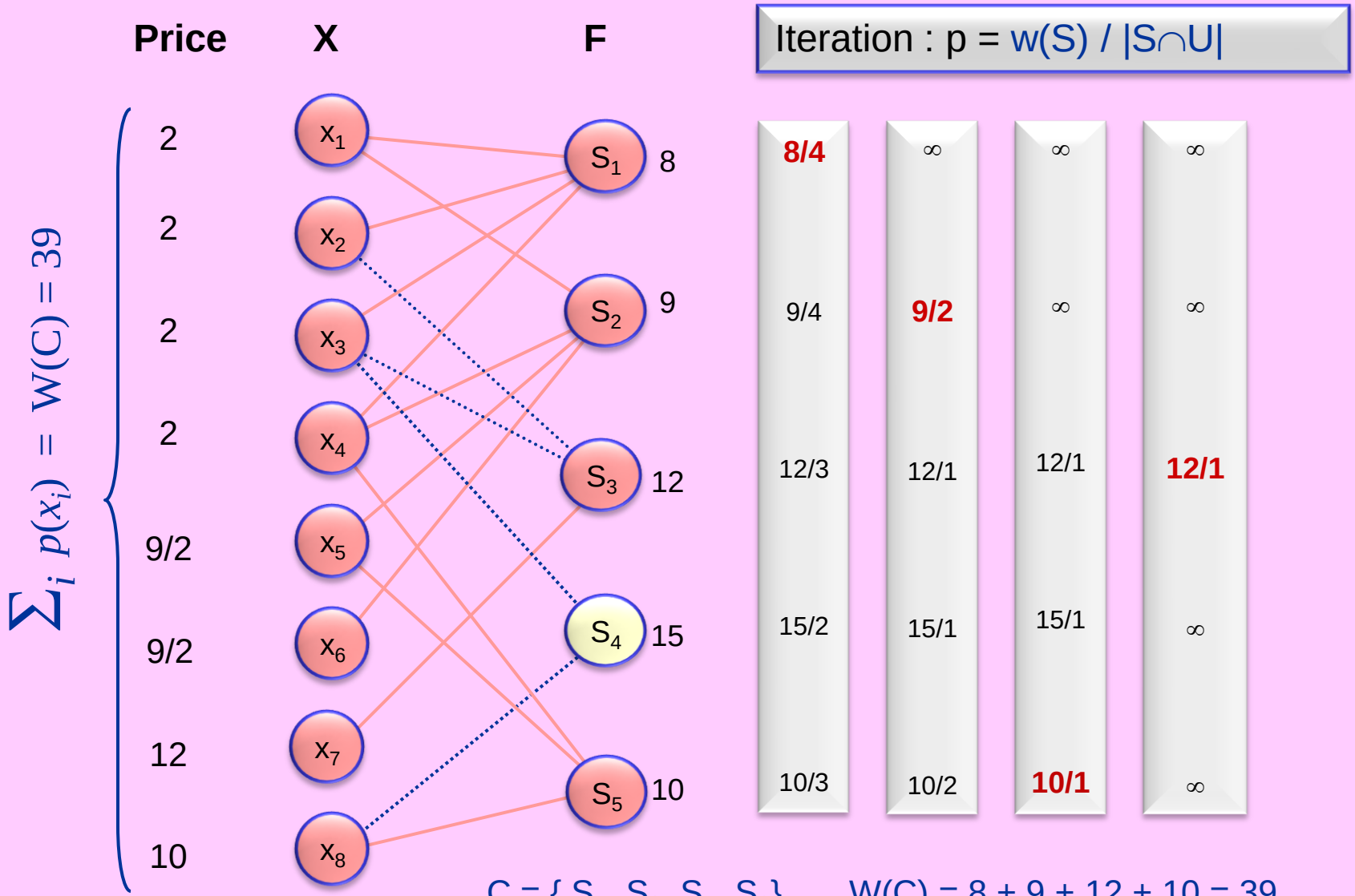
Example run of the algorithm



Iteration : p = w(S) / |S∩U|

8/4	∞	∞	∞
9/4	**9/2**	∞	∞
12/3	12/1	12/1	**12/1**
15/2	15/1	15/1	∞
10/3	10/2	**10/1**	∞

Example run of the algorithm



$C = \{ S_1, S_2, S_5, S_3\}$, $W(C) = 8 + 9 + 12 + 10 = 39$

$C_{OPT} = \{S_2, S_3, S_5\}$, $W(C_{OPT}) = 9 + 12 + 10 = 31$

This is an H(n)-approximation algorithm

$$\text{Harmonic Number}: \quad H(d) = \sum_{i=1}^{d} \frac{1}{i} = 1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{d} \leq \ln d + 1.$$

$$\text{Maximum degree}: \quad d_{max} = \max\{ |S| : S \in F\} \leq |X| = n$$

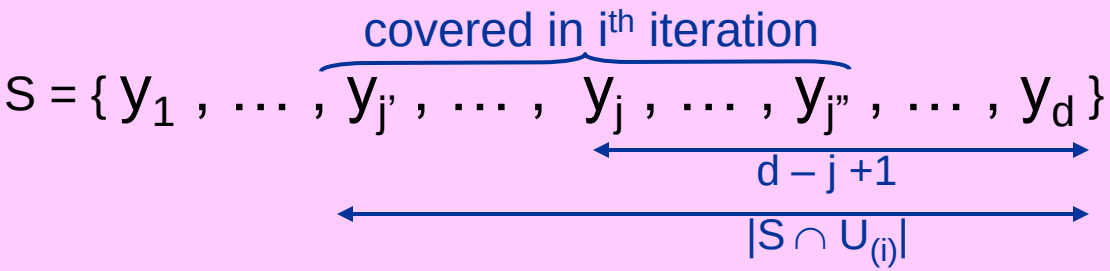
$$H(d_{max}) \leq H(n).$$

This is an H(n)-approximation algorithm

LEMMA: $\forall S \in F \quad \sum_{x \in S} p(x) \le w(S)\, H(|S|)$.

Proof: Let $d = |S|$ & $S = \{ y_1 , y_2 , \ldots , y_d \}$ elements in order covered by algorithm (break ties arbitrarily).

Each $y_j \in S$ is covered during some (earliest) iteration i.
$S_{(i)}$ = set selected by algorithm during i^{th} iteration (similarly, $U_{(i)}$).



$$p(y_j) = \frac{w(S_{(i)})}{|S_{(i)} \cap U_{(i)}|} \underset{\text{greedy selection}}{\le} \frac{w(S)}{|S \cap U_{(i)}|} \le \frac{w(S)}{d-j+1}$$

$$\sum_{x \in S} p(x) = \sum_{j=1}^{d} p(y_j) \le \sum_{j=1}^{d} \frac{w(S)}{d-j+1} = w(S)\left(\tfrac{1}{d} + \tfrac{1}{d-1} + \cdots + \tfrac{1}{2} + 1\right) = w(S)\, H(|S|)$$

This is an H(n)-approximation algorithm

THEOREM: The Greedy-Set-Cover algorithm has the following properties:

(1) Correctness: outputs a feasible set cover C
(2) Running Time: polynomial
(3) Approx Bound: $W(C) \le H(d_{max})\, W(C_{OPT})$
$\le H(n)\, W(C_{OPT})$

Proof: (1) & (2) are obvious. Proof of (3):

$$W(C) \underset{\substack{\text{each x}\\ \text{priced}\\ \text{once}\\ \text{in cover C}}}{=} \sum_{x \in X} p(x) \underset{\substack{\text{each x}\\ \text{covered}\\ \text{by at least}\\ \text{one set}\\ \text{in } C_{OPT}}}{\le} \sum_{S \in C_{OPT}} \left(\sum_{x \in S} p(x) \right) \underset{\substack{by\\ LEMMA}}{\le} \sum_{S \in C_{OPT}} w(S)\, H(|S|)$$

$$\le H(d_{\max}) \sum_{S \in C_{OPT}} w(S) = H(d_{\max}) W(C_{OPT}).$$

Therefore $\dfrac{W(C)}{W(C_{OPT})} \le H(d_{\max}) \le H(|X|) = H(n).$

This is an H(n)-approximation algorithm

THEOREM: The Greedy-Set-Cover algorithm has the following properties:

(1) Correctness: outputs a feasible set cover C
(2) Running Time: polynomial
(3) Approx Bound: $W(C) \leq H(d_{max})\, W(C_{OPT})$
$\leq H(n)\, W(C_{OPT})$

An alternative proof of (3):

- From the Lemma we have $\sum_{x \in S} p(x) \leq w(S)\, H(|S|) \leq w(S) H(d_{max}) \quad \forall S \in F$.
- So the following scaled down prices are feasible in the dual of the LP relaxation:

$$p'(x) = \frac{p(x)}{H(d_{max})} \quad \forall x \in X.$$

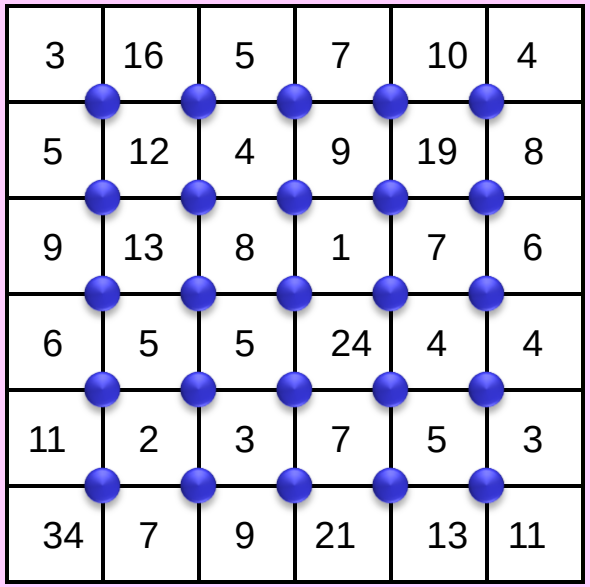
- So, dual cost of this feasible solution is a lower-bound on the optimal ILP cost:

$$\sum_{x \in X} p'(x) \leq W(C_{OPT}).$$

- We also have $\sum_{x \in X} p'(x) = \frac{\sum_{x \in X} p(x)}{H(d_{max})} = \frac{W(C)}{H(d_{max})}$.
- We conclude $\frac{W(C)}{W(C_{OPT})} \leq H(d_{max})$.

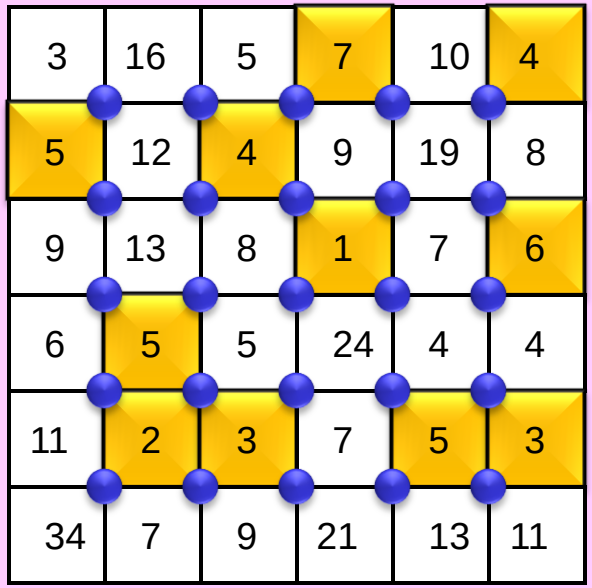
Problem 1

Input: Weighted n×n grid squares.
Output: A subset of grid squares that collectively cover all inner grid corners (i.e., each ● is incident to at least one selected grid square).
Goal: Minimize weight-sum of selected grid squares.



Problem 1

Input: Weighted n×n grid squares.
Output: A subset of grid squares that collectively cover all inner grid corners (i.e., each ● is incident to at least one selected grid square).
Goal: Minimize weight-sum of selected grid squares.



ρ–approximation by pricing á la:

1) Vertex Cover (on hyper-graph): $\rho = 4$

2) Set Cover: $\rho = H(4) = 2.083\ldots$

Question: Is this special case of the Set Cover Problem still NP-hard? Can it be solved in poly-time (say, by dynamic programming)?

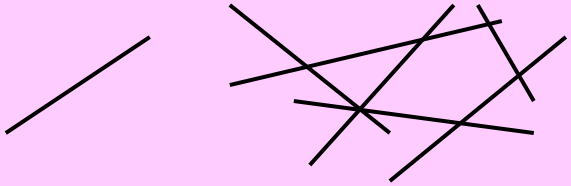
Problems 2 & 3

Problem 2:

Input: A set S of n line segments in the plane.

Output: A minimum number of points P in the plane that collectively "hit" S (i.e., every segment in S has some point in P).

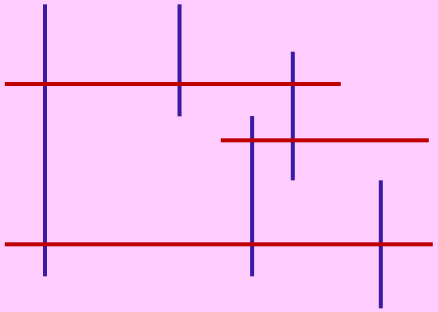
Example:



This special case of the set cover problem is still NP-hard. Any better approximation algorithm?

Problem 3: Same problem, only vertical and horizontal line segments.

Example:



Is this in P or still NP-hard? Any better approximation algorithm? Bipartite matching?

Traveling Salesman Problem

Traveling Salesman Problem (TSP)

Input: An $n \times n$ positive distance matrix $D=(d_{ij})$, $i,j = 1..n$, where d_{ij} is the travel distance from city i to city j.

Output: A traveling salesman tour T. T starts from the home city (say, city 1) and visits each city exactly once and returns to home city.

Goal: minimize total distance traveled on tour T: $C(T) = \sum_{(i,j)\in T} d_{ij}$.

$$D = \begin{bmatrix} 0 & 7 & 2 & 11 \\ 5 & 0 & 1 & 9 \\ 3 & 2 & 0 & 5 \\ 12 & 3 & 9 & 0 \end{bmatrix}$$

$T_{OPT} \equiv (1, 3, 4, 2) \equiv ((1, 3), (3,4), (4,2), (2,1))$

$C(T_{OPT}) = d_{13} + d_{34} + d_{42} + d_{21}$

$= 2 + 5 + 3 + 5$

$= 15$

Some Classes of TSP

- **General TSP :** distance matrix D is arbitrary
- **Metric-TSP:** D satisfies the metric axioms
- **Euclidean-TSP:** n cities as n points in the plane with Euclidean inter-city distances

These are all NP-hard.

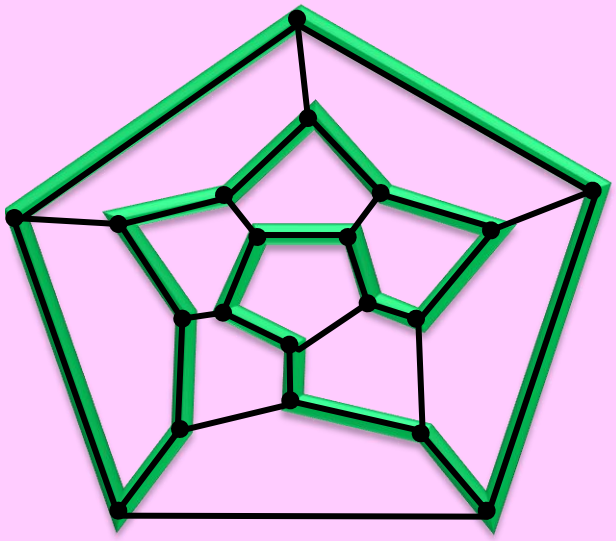
Related Problems:

- Minimum Spanning Tree
- Hamiltonian Cycle
- Graph Matching
- Eulerian Graphs

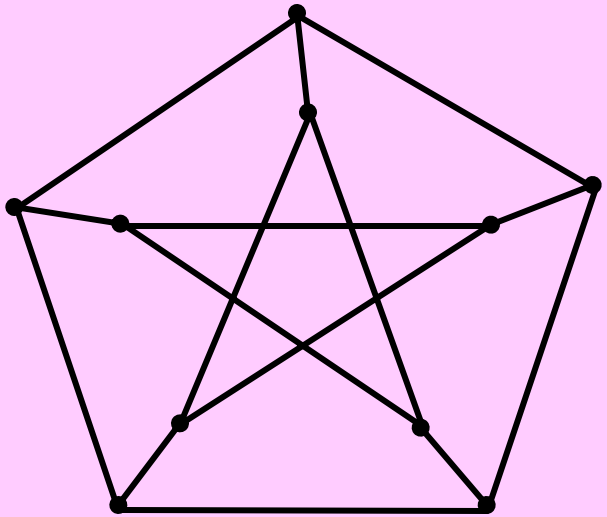
Hamiltonian Cycle Problem (HCP)

HCP: Given a graph G(V,E), does G have a Hamiltonian Cycle (HC)?
HC is any simple spanning cycle of G, i.e., a cycle that goes through each vertex exactly once.

HCP is known to be NP-hard.



Hamiltonian
(skeleton of dodecahedron)



Non-Hamiltonian
(Peterson graph)

General TSP

THEOREM: Let $\rho > 1$ be any constant.
ρ-approximation of general TSP is also NP-hard.
[So, there is no polynomial-time ρ-approximation algorithm for general TSP, unless P=NP.]

Proof: Reduction from Hamiltonian Cycle Problem (HCP).
HCP: Given G(V,E), is G Hamiltonian?

Reduction: Let n = |V|. Define the TSP distance matrix as:

$$d_{ij} = \begin{cases} 1 & \text{if } (i,j) \in E \\ 1 + \lceil \rho n \rceil & \text{if } (i,j) \notin E \end{cases}$$

G is Hamiltonian $\Leftrightarrow$ $C(T_{OPT}) = n$.
G is not Hamiltonian $\Leftrightarrow$ $C(T_{OPT}) \geq n + \lceil \rho n \rceil \geq (1+\rho)\, n > \rho\, n$.

Suppose there were a poly-time ρ-approx TSP algorithm producing a tour T.
$C(T)/C(T_{OPT}) \leq \rho \Rightarrow$ G has HC if and only if $C(T) = n$.

This would provide a poly-time algorithm for the HCP!

Metric & Euclidean TSP

- **Metric Traveling Salesman Problem (metric-TSP):**
  - special case of general TSP (distance matrix is metric)
  - NP-hard
  - 2-approximation: Rosenkrants-Stearns-Lewis [1974]
  - 1.5-approximation: Christofides [1976]

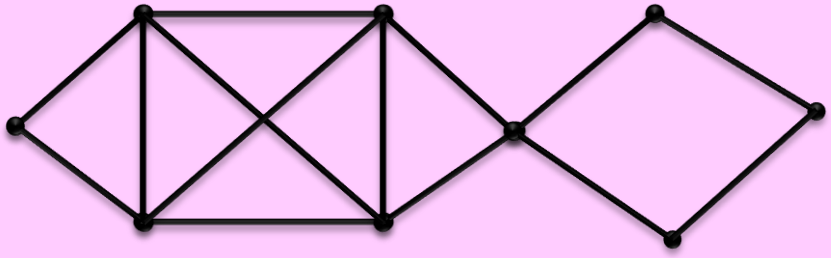
- **Euclidean Traveling Salesman Problem (ETSP):**
  - special case of Metric-TSP (with Euclidean metric)
  - NP-hard
  - PTAS: Sanjeev Arora [1996] [Lecture Note 10].

Eulerian Graph

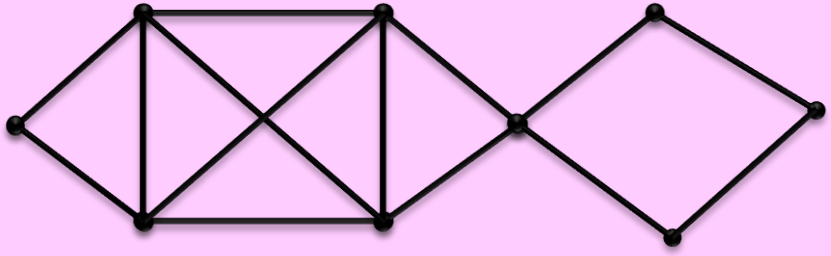
Definition: A graph G(V,E) is **Eulerian** if it has an Euler walk.

Euler Walk is a cycle that goes through each **edge** of G exactly once.

An Eulerian graph G



An Euler walk of G



FACT 1: A graph G is Eulerian if and only if

(a) G is connected and

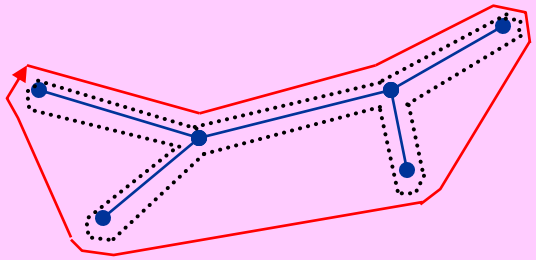
(b) every vertex of G has even degree.

FACT 2: An Euler walk of an Eulerian graph can be found in linear time.

2-approximation of metric-TSP

Rosenkrants-Stearns-Lewis [1974] (See also AAW)

$$C(T) \leq 2 \times C(T_{OPT})$$

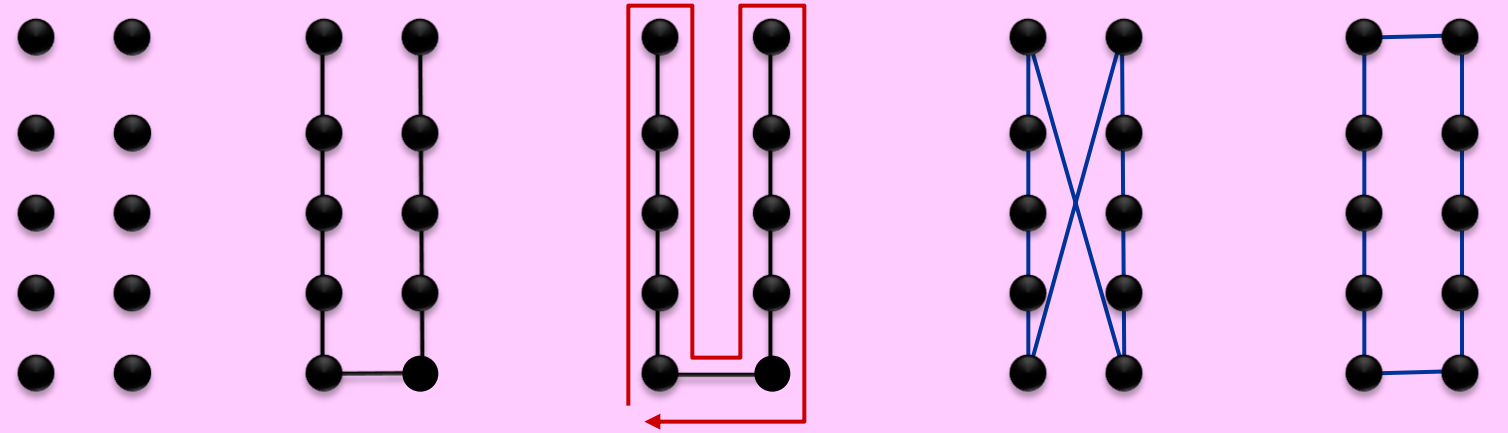


- Minimum Spanning Tree (MST)
- Euler walk around double-MST
- Bypass repeated nodes on the Euler walk.

FACT 1: Triangle inequality implies bypassing nodes cannot increase length of walk.

FACT 2: **LB =** $C(MST) \leq C(T_{OPT}) \leq C(T)$ **= UB** $\leq 2 \times C(MST)$ **= 2 LB**.

$\rho = 2$ is tight (even for Euclidean instances)



Euclidean Instance

MST

Euler walk of double MST

Tour T

Tour T_{OPT}

$$C(T_{OPT}) = n$$

$$C(T) = 2\left(\tfrac{n}{2} - 1 + \sqrt{(\tfrac{n}{2} - 1)^2 + 1}\right) = 2n - O(1)$$

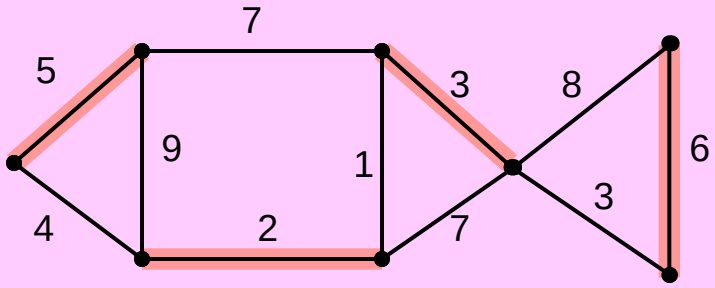
$$\rho(n) = \frac{C(T)}{C(T_{OPT})} = 2 - O\left(\tfrac{1}{n}\right)$$

$$\lim_{n \to \infty} \rho(n) = 2$$

Graph Matching

Definition:

- **A matching M** in a graph G(V,E) is a subset of the edges of G such that no two edges in M are incident to a common vertex.
- **Weight** of M is the sum of its edge weights.
- **A perfect** matching is one in which every vertex is matched.



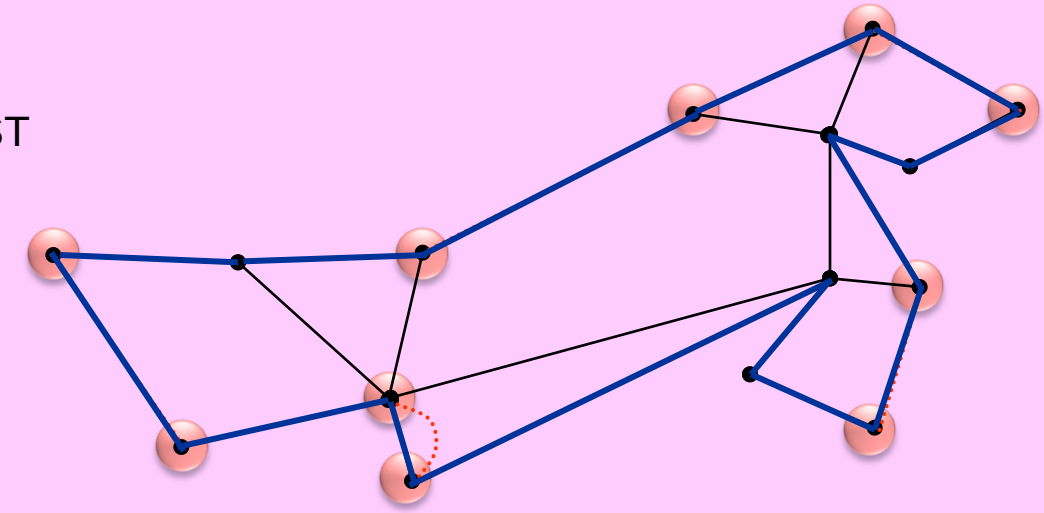
A perfect matching M of weight 5+2+3+6=16 in graph G.

FACT: Minimum weight maximum cardinality matching can be obtained in polynomial time [Jack Edmonds 1965].

1.5-approximation for metric-TSP

Christofides [1976]: $C(T) \le 1.5 \times C(T_{OPT})$

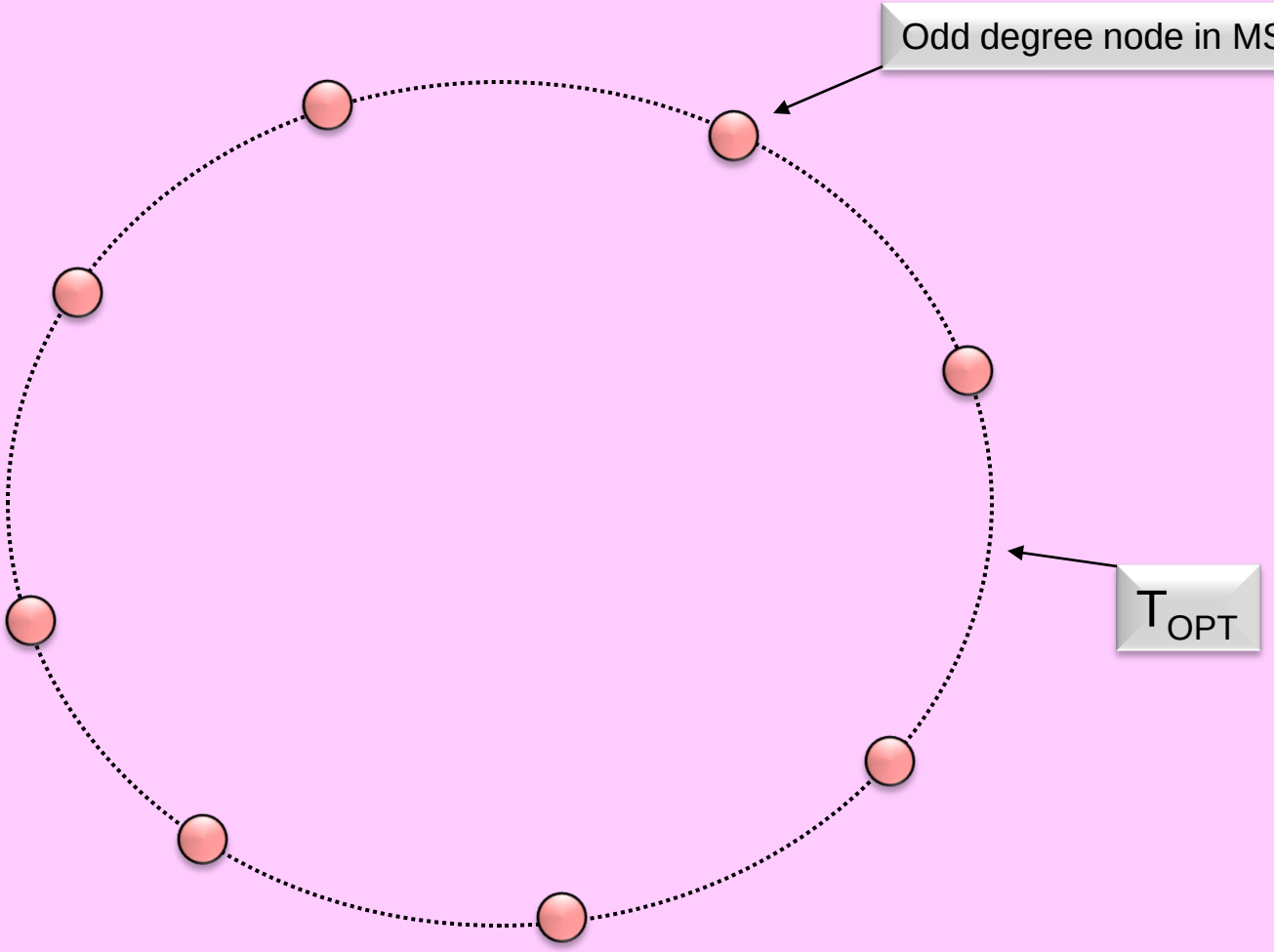
- MST
- Odd degree nodes in MST
- M = Minimum weight perfect matching on odd-degree nodes
- E = MST + M is Eulerian. Find an Euler walk of E.
- Bypass repeated nodes on the Euler walk to get a TSP tour T.



FACTS:

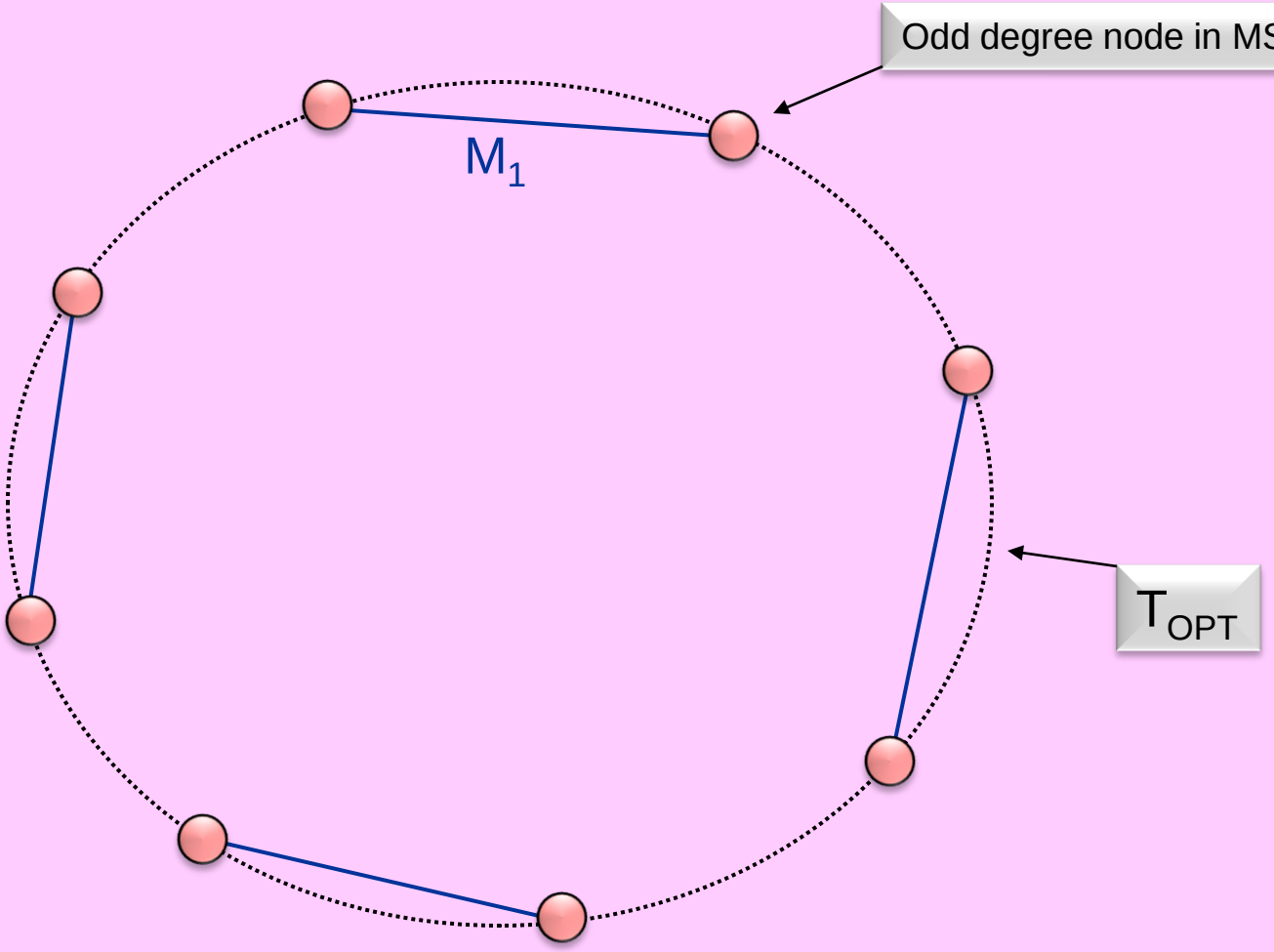
- Any graph has even # of odd degree nodes. (Hence, M exists.)
- $C(MST) \le C(T_{OPT})$
- $C(M) \le 0.5 \times C(T_{OPT})$ (See next slide.)
- $C(E) = C(MST) + C(M) \le 1.5 \times C(T_{OPT})$
- $C(T) \le C(E) \le 1.5 \times C(T_{OPT})$

$$C(M) \leq 0.5 \times C(T_{OPT})$$



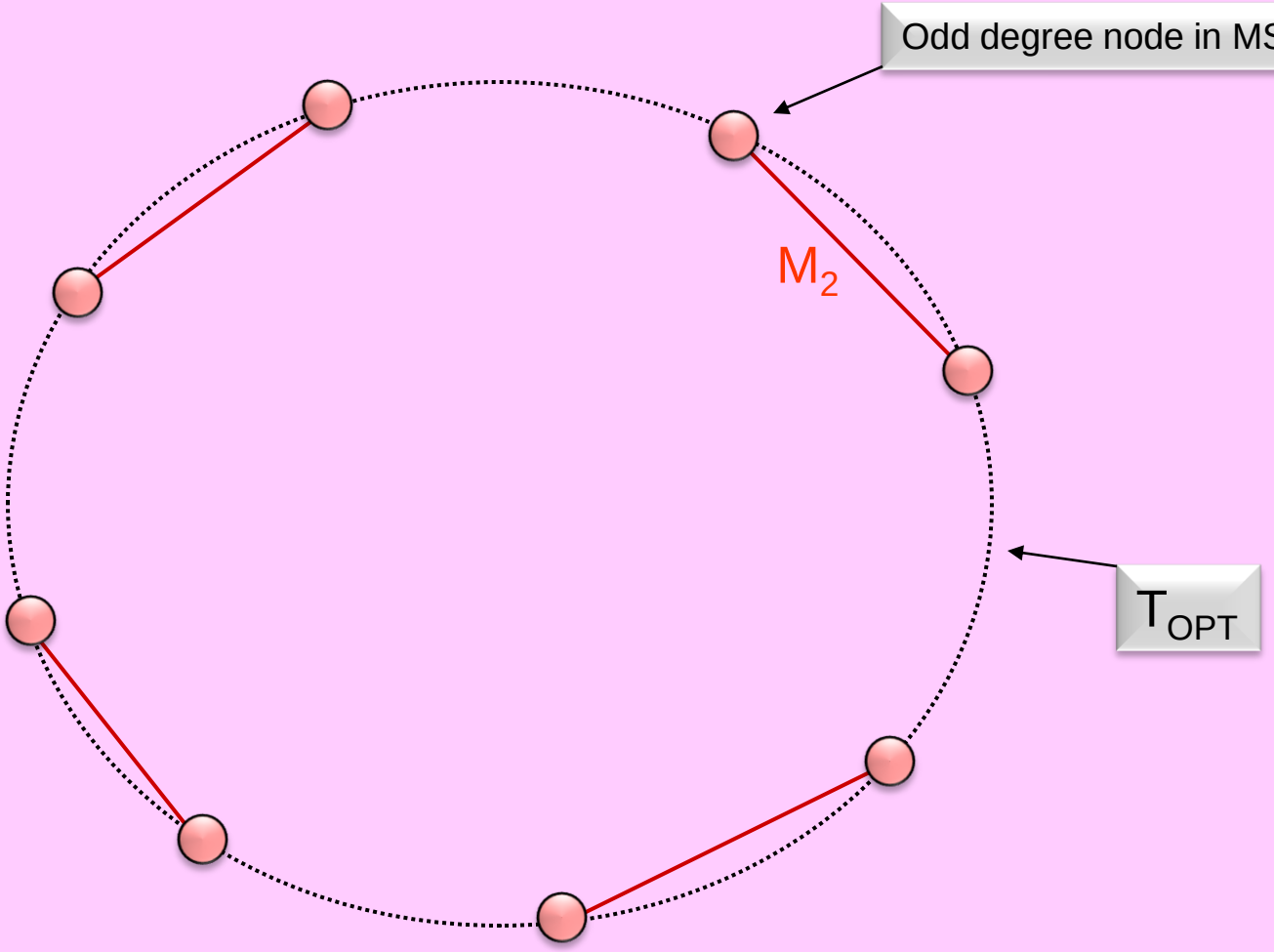
$C(M) \leq 0.5 \times C(T_{OPT})$

$C(M) \leq C(M_1)$



$$C(M) \leq 0.5 \times C(T_{OPT})$$

$C(M) \leq C(M_2)$

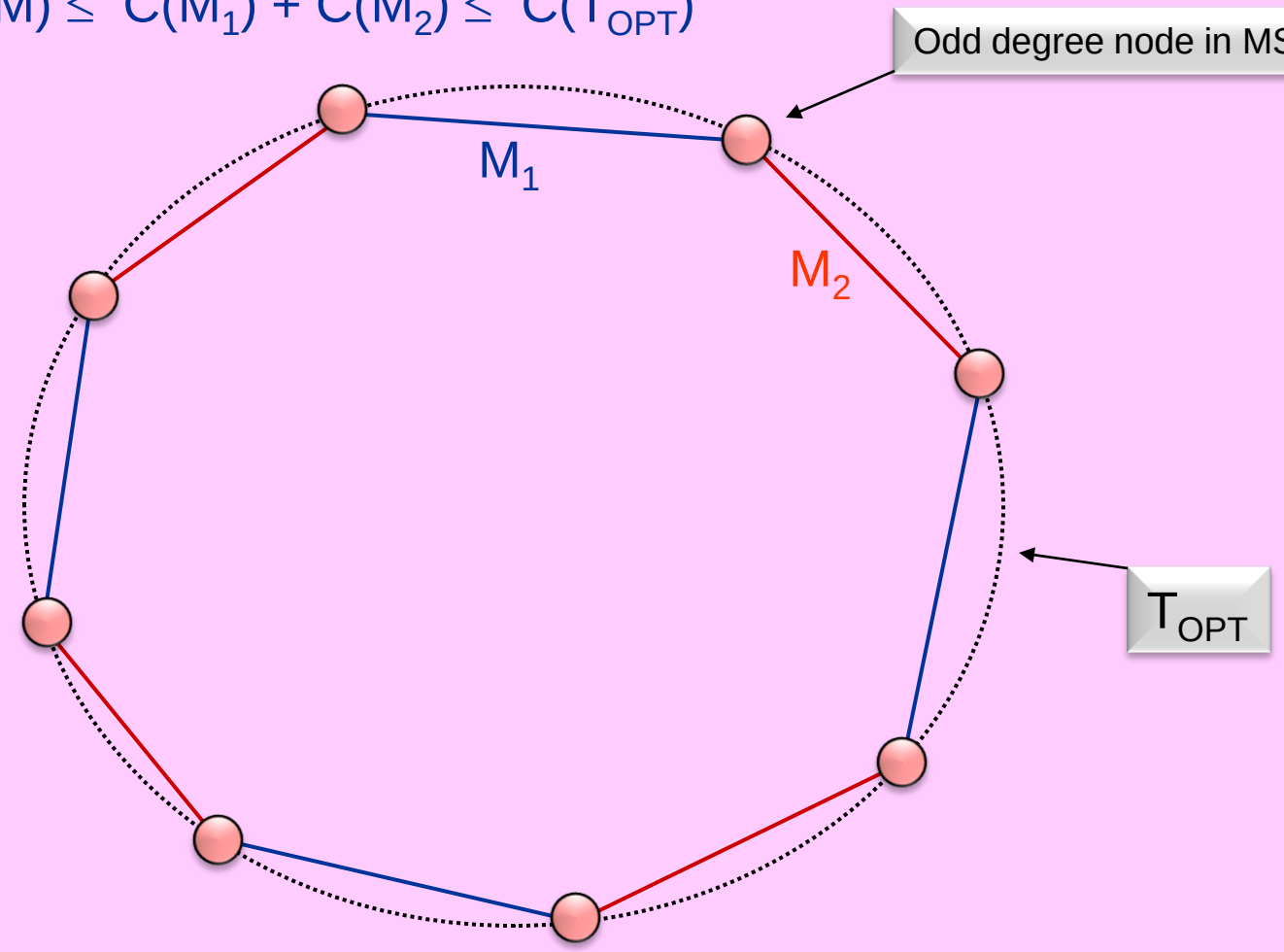


$C(M) \leq 0.5 \times C(T_{OPT})$

$C(M) \leq C(M_1)$

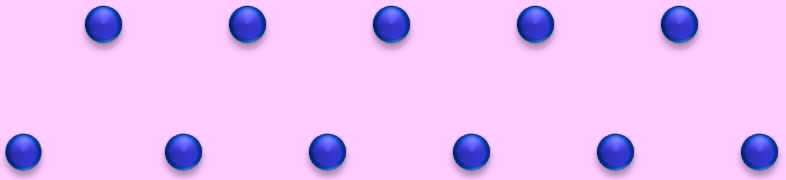
$C(M) \leq C(M_2)$

$2 \times C(M) \leq C(M_1) + C(M_2) \leq C(T_{OPT})$

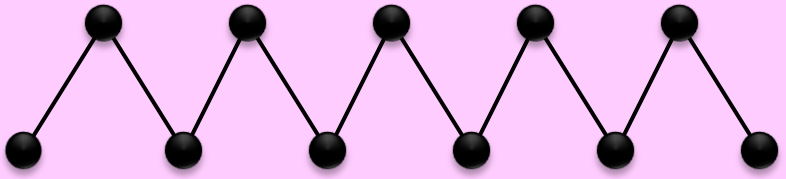


$\rho = 1.5$ is tight (even for Euclidean instances)

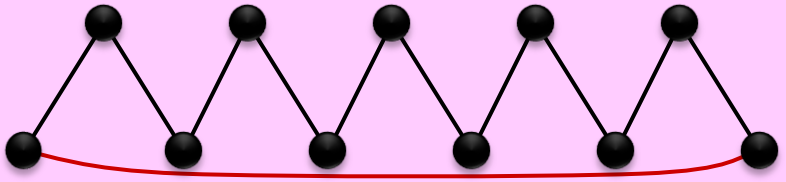
Euclidean instance:



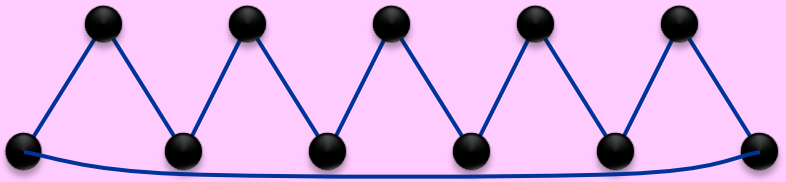
MST:



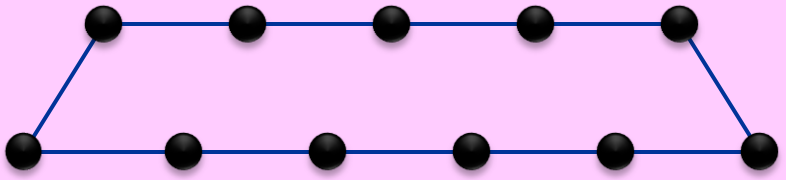
MST + M:



Tour T:



Tour T_{OPT} :



TSP: some recent developments

- Jeff Jones and Andrew Adamatzky,

  "Computation of the Travelling Salesman Problem by a Shrinking Blob",

  URL: http://arxiv.org/pdf/1303.4969v2.pdf, March 2013.

- Hyung-Chan An, Robert Kleinberg, David B. Shmoys,

  "Improving Christofides' Algorithm for the s-t Path TSP" ,

  URL: http://arxiv.org/pdf/1110.4604v2.pdf, May 2012.

K-Cluster Problem

The K-Cluster Problem

Input: Points $X = \{x_1, x_2, \ldots, x_n\}$ with underlying distance metric $d(x_i, x_j)$, i,j=1..n, and positive integer K.

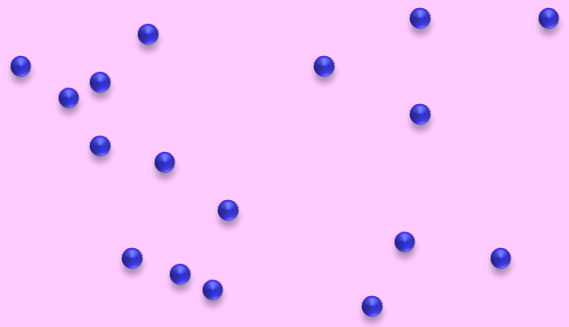
Output: A partition of X into K clusters $C_1, C_2, , \ldots, C_k$.

Goal: minimize the longest diameter of the clusters:

$$\max_j \max_{a,b \in C_j} d(a,b).$$

An Euclidean version: given n points in the plane, find K equal & minimum diameter circular disks that collectively cover the n points. This Euclidean version is also NP-hard.

n=17 points



The K-Cluster Problem

Input: Points $X = \{x_1, x_2, \ldots, x_n\}$ with underlying distance metric $d(x_i, x_j)$, i,j=1..n, and positive integer K.

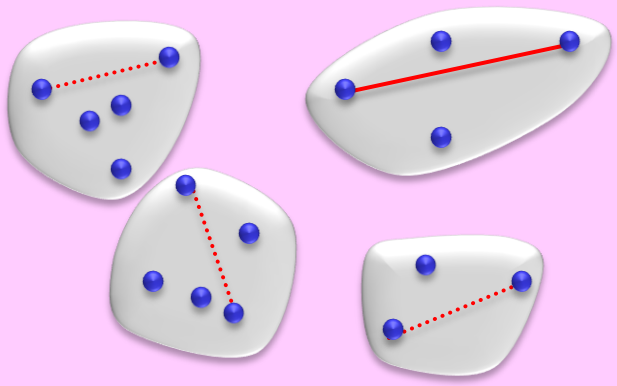
Output: A partition of X into K clusters $C_1, C_2, , \ldots, C_k$.

Goal: minimize the longest diameter of the clusters:

$$\max_{j} \max_{a,b \in C_j} d(a,b).$$

An Euclidean version: given n points in the plane, find K equal & minimum diameter circular disks that collectively cover the n points. This Euclidean version is also NP-hard.

n=17 points in K=4 clusters:



Greedy Approximation

IDEA: (1) Pick K points $\{ \mu_1 , \mu_2 , \ldots , \mu_K \}$ from X as cluster "centers".
Greedily & incrementally pick cluster centers,
each **farthest** from the previously selected ones.
(2) Assign remaining points of X to the cluster with **closest** center.
(Break ties arbitrarily.)

ALGORITHM Approximate-K-Cluster (X, d, K)

Pick any point $\mu_1 \in X$ as the first cluster center

for $i \leftarrow 2 .. K$ **do**
Let $\mu_i \in X$ be the point farthest from $\{ \mu_1 , \ldots , \mu_{i-1} \}$
(i.e., μ_i maximizes $r(i) = \min_{j<i} d(\mu_i , \mu_j)$)

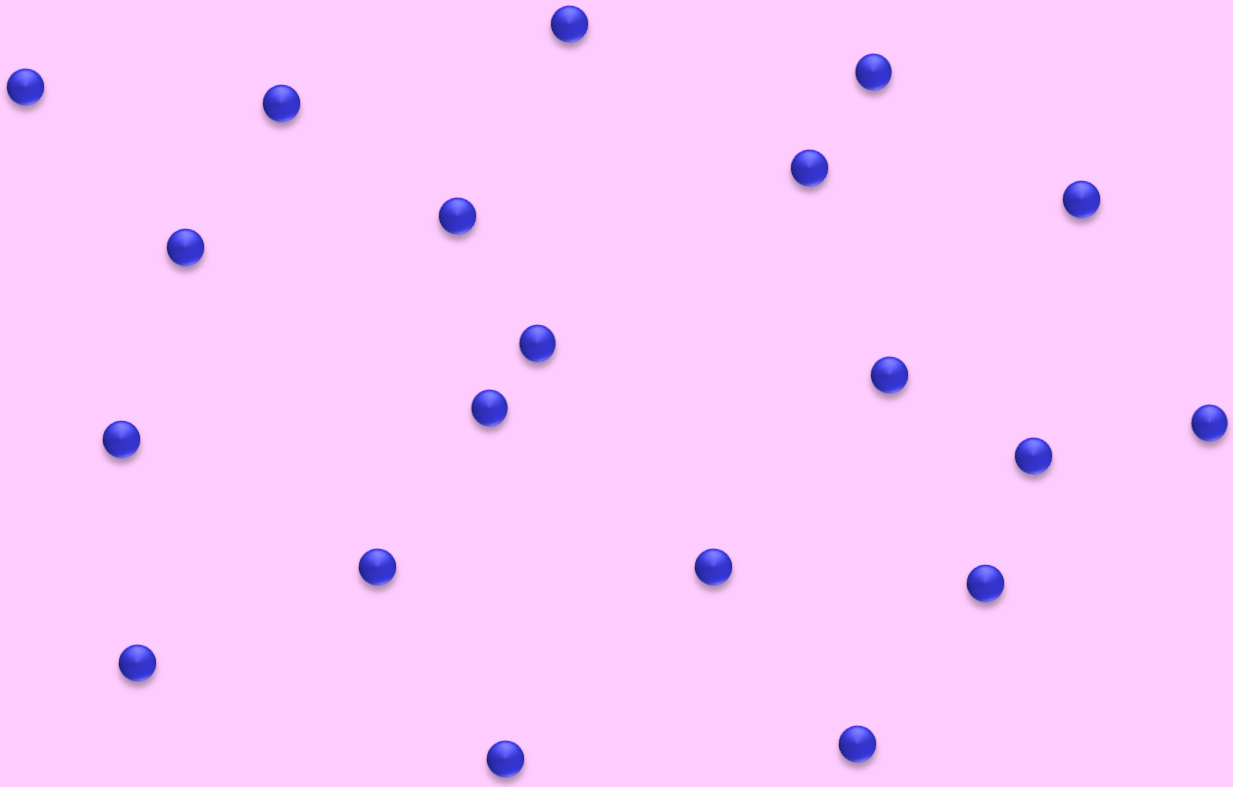
for $i \leftarrow 1 .. K$ **do**
$C_i \leftarrow \{ x \in X \mid \mu_i$ is the closest center to x $\}$ (* break ties arbitrarily *)

return the K clusters $\{ C_1, C_2 , \ldots , C_k \}$

end

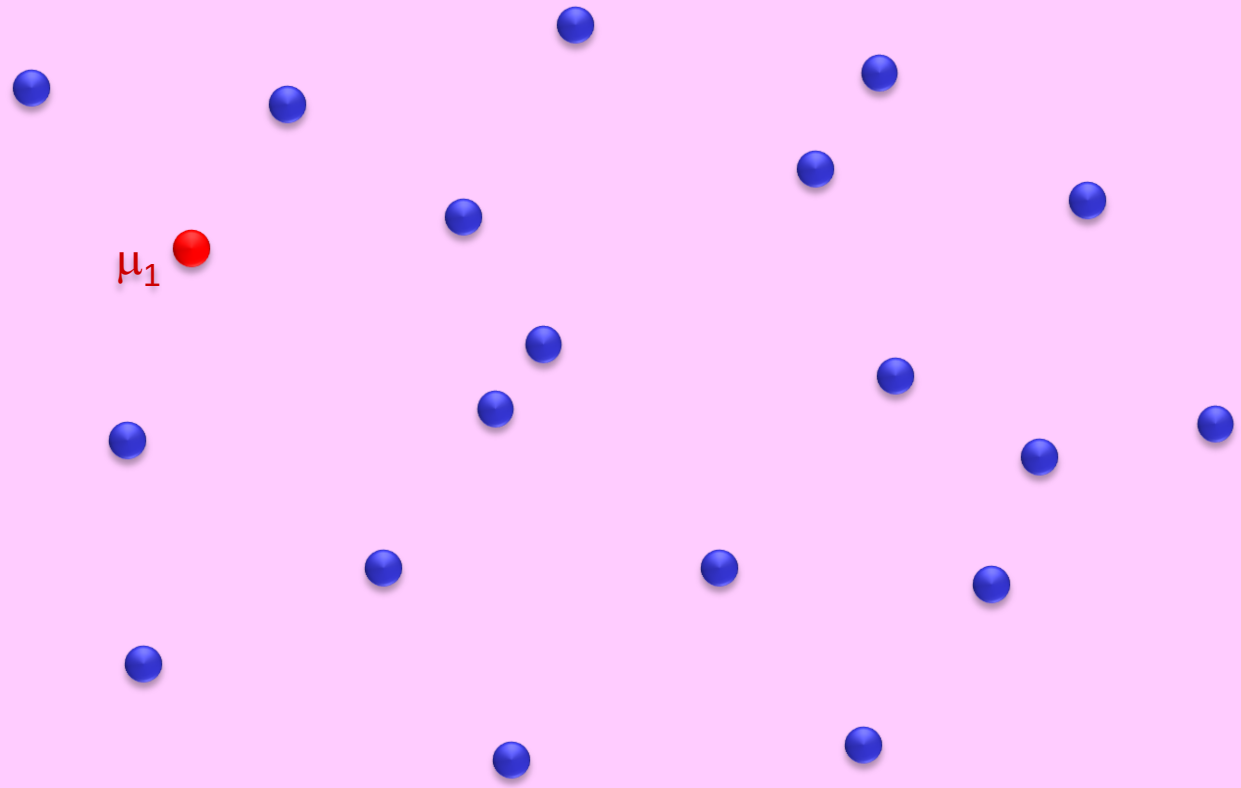
Greedy Approximation Example

n = 20, K = 4



Greedy Approximation Example

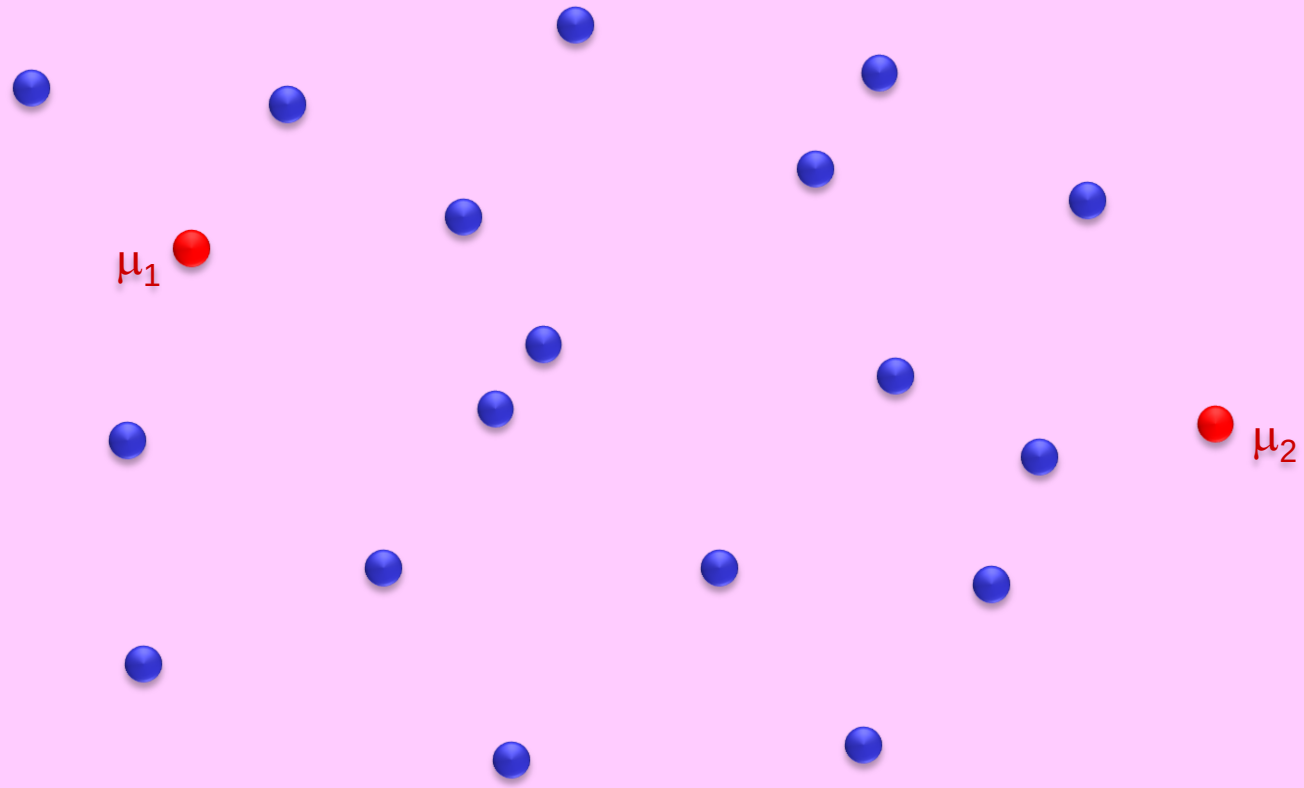
n = 20, K = 4



Pick the first center arbitrarily

Greedy Approximation Example

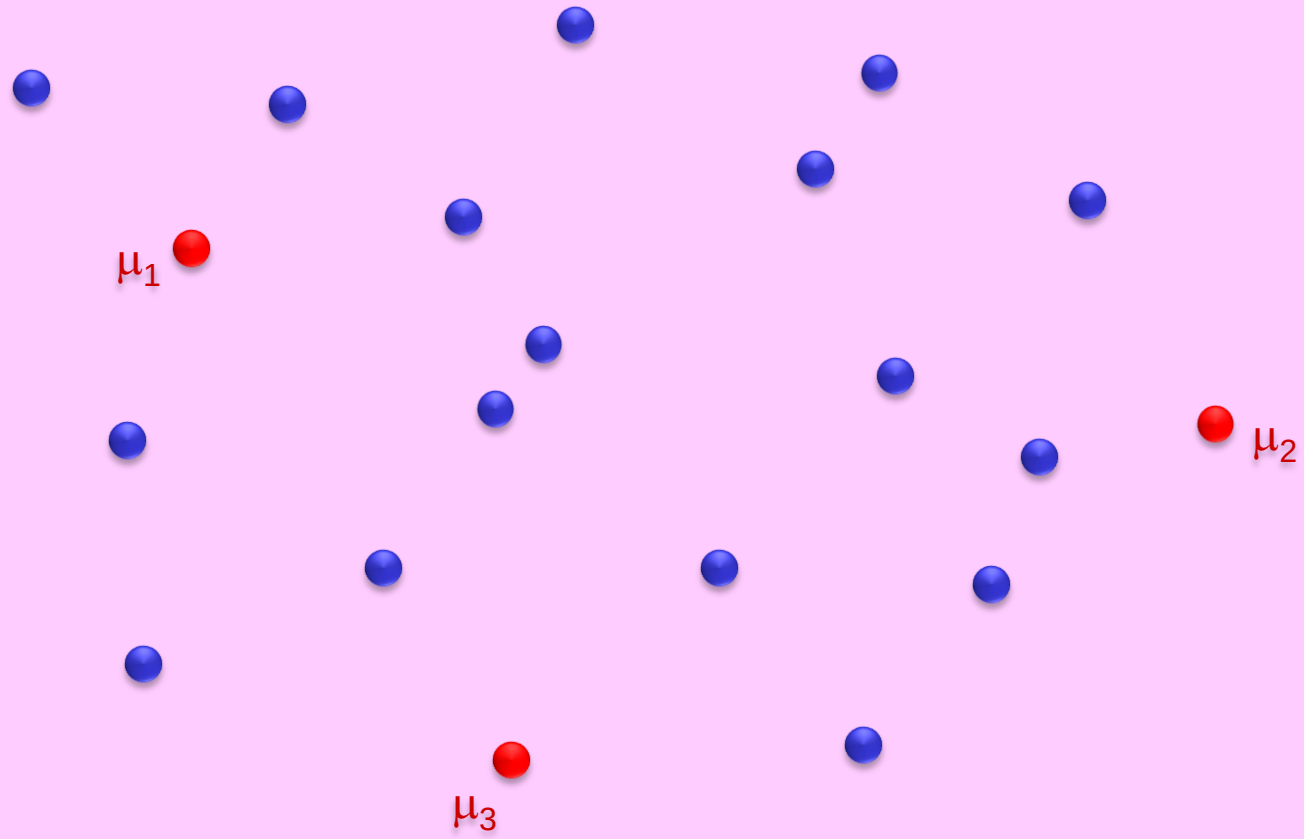
n = 20, K = 4



Pick the next center farthest from previous ones

Greedy Approximation Example

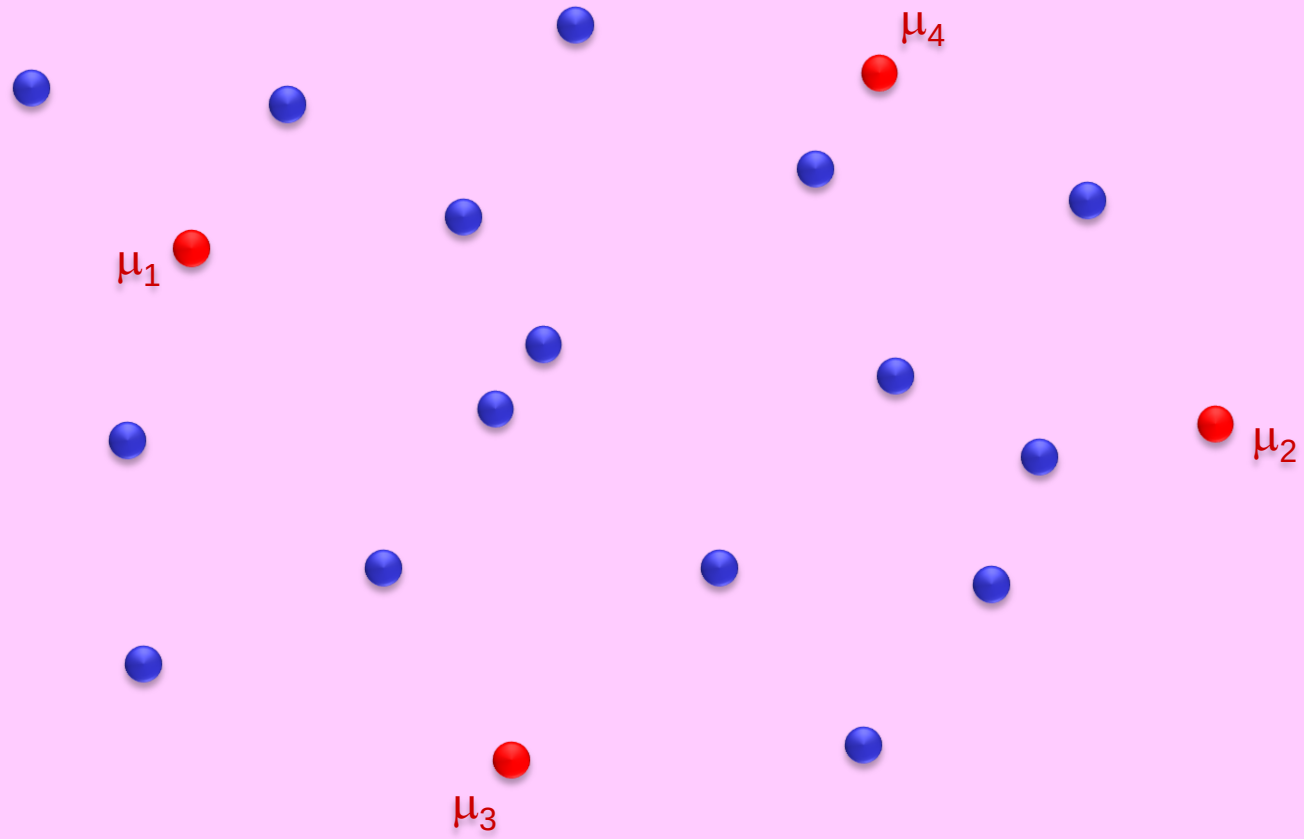
n = 20, K = 4



Pick the next center farthest from previous ones

Greedy Approximation Example

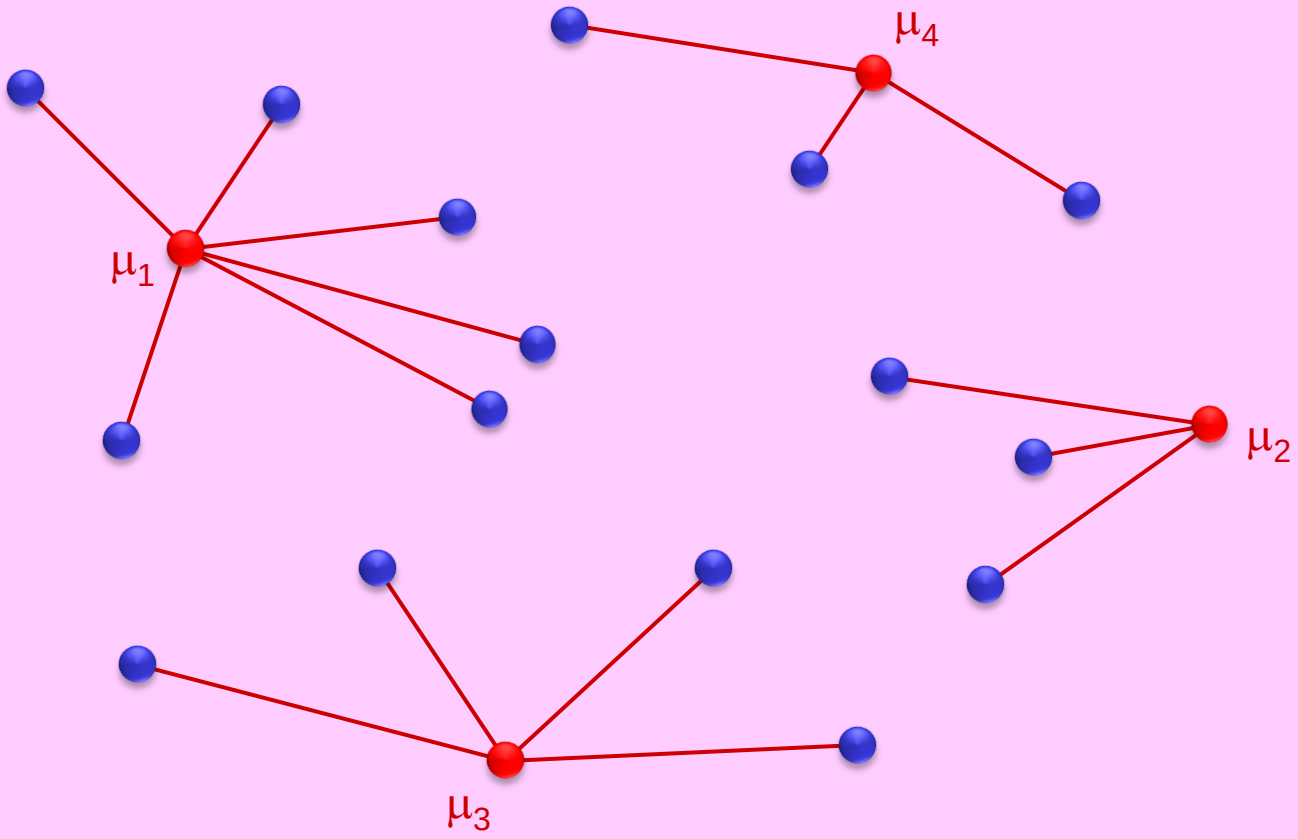
n = 20, K = 4



Pick the next center farthest from previous ones

Greedy Approximation Example

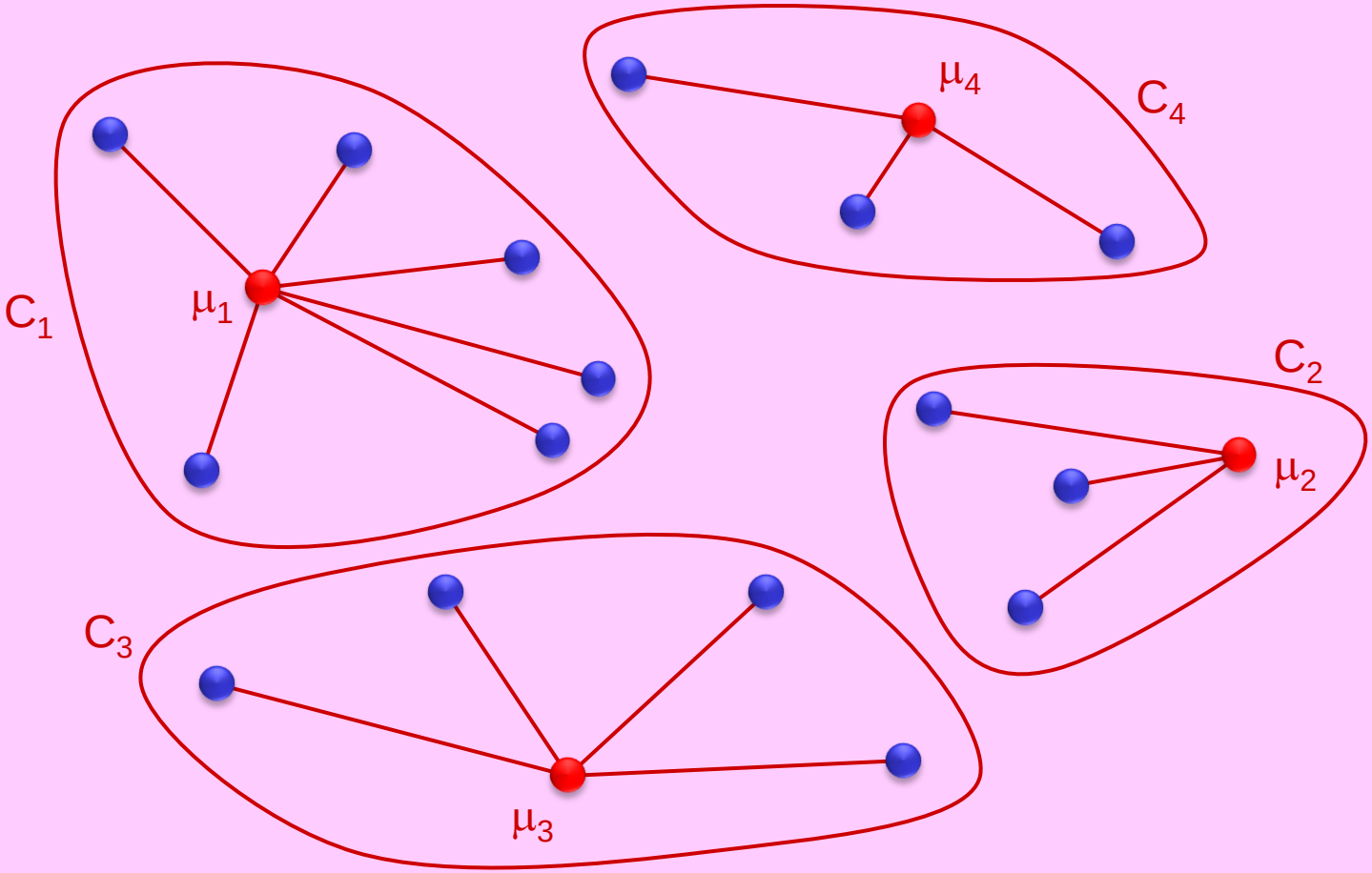
n = 20, K = 4



Assign each point to its closest center

Greedy Approximation Example

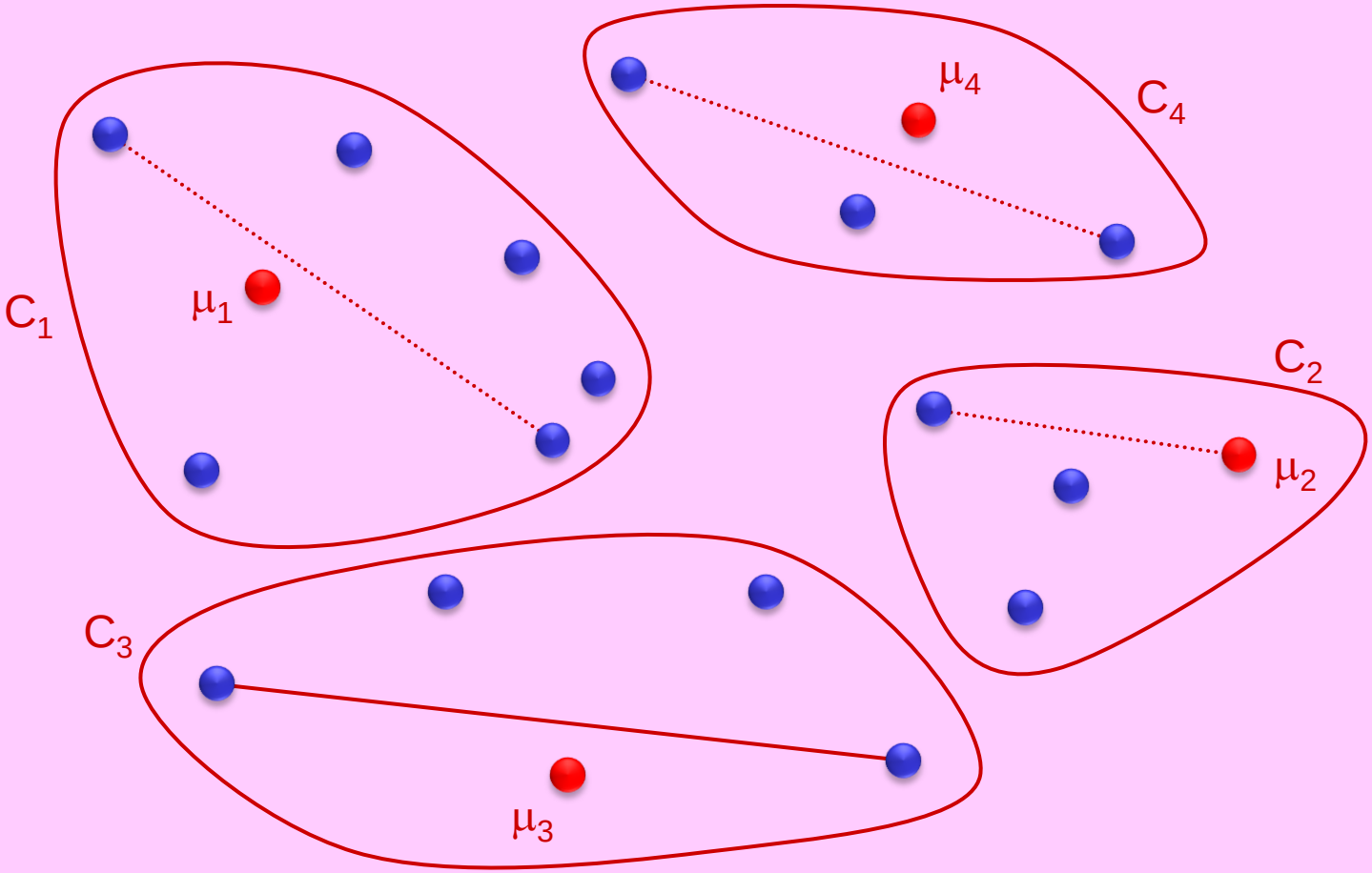
n = 20, K = 4



Form K clusters

Greedy Approximation Example

n = 20, K = 4



Objective cost = largest cluster diameter

This is a 2-approximation algorithm

Definition: Let $x^* \in X$ be the point farthest from $\{ \mu_1 , \mu_2 , \dots , \mu_K \}$,
i.e., $x^* = \mu_{K+1}$, if we wanted K+1 centers.
Let $r^* = r(K+1) = \min \{ d(x^* , \mu_j) \mid j = 1 .. K \}$.

LEMMA: The algorithm has the following properties:
(a) Every point is within distance at most r^* of its cluster center.
(b) The K+1 points $\{ \mu_1 , \mu_2 , \dots , \mu_K , \mu_{K+1}=x^* \}$ are all at a distance at least r^* from each other.

Proof: $r(i) = \min \{ d(\mu_i , \mu_j) \mid j = 1 .. i-1 \}$, is a monotonically decreasing function of i. …

THEOREM: Approximate-K-Cluster is a 2-approx poly-time algorithm.

Proof: (1) $a,b \in C_i \Rightarrow d(a,b) \le d(a, \mu_i) + d(\mu_i,b) \le 2r^*$ (by Lemma(a))
Thus, **UB =** $\max \{ \text{diameter}(C_i) \mid i=1..K \} \le 2r^*$.

(2) Pigeon-Hole Principle: In the OPT K-Clustering, for some $i \neq j \in \{1,\dots,K+1\}$, μ_i and μ_j will be placed in the same OPT cluster.
Lemma (b) $\Rightarrow$ diameter of that OPT cluster $\ge r^*$ **= LB ≥ ½ UB.**

0/1 Knapsack Problem

0/1 Knapsack Problem

Input: n items with weights $w_1, w_2, \ldots, w_n$ and values $v_1, v_2, \ldots, v_n$, and knapsack weight capacity W (all positive integers).

Output: A subset S of the items (to be placed in the knapsack) whose total weight does not exceed the knapsack capacity.

Goal: maximize the total value of items in S: $V(S) = \sum_{i \in S} v_i$.

[CLRS] considers a special case of this problem called the Subset Sum Problem. In SSP, $w_i = v_i$, for i=1..n. Both problems are NP-hard.

(01KP) ILP formulation :

$$\text{maximize} \quad \sum_{i=1}^{n} v_i x_i$$

$$\text{subject to :} \quad (1) \quad \sum_{i=1}^{n} w_i x_i \le W$$

$$(2) \quad x_i \in \{0,1\} \quad \text{for } i = 1..n.$$

WLOG assume :

(a) $\forall i \; w_i \le W$

(b) $\sum_{i=1}^{n} w_i > W$

0/1 vs Fractional KP

(01KP) :

$$\text{maximize} \quad \sum_{i=1}^{n} v_i x_i$$

$$\text{subject to :} \quad (1) \quad \sum_{i=1}^{n} w_i x_i \le W$$

$$(2) \quad x_i \in \{0,1\} \qquad \text{for } i = 1..n.$$

(FKP) as LP relaxation of 01KP :

$$\text{maximize} \quad \sum_{i=1}^{n} v_i x_i$$

$$\text{subject to :} \quad (1) \quad \sum_{i=1}^{n} w_i x_i \le W$$

$$(2') \quad 0 \le x_i \le 1 \qquad \text{for } i = 1..n.$$

FACT: 01KP $V_{OPT} \le$ FKP V_{OPT} .

Algorithmic Results

1. An exponential-time exact algorithm for 01KP by dynamic programming.

   In EECS3101 (LS7) we showed such a DP algorithm by dynamizing on aggregate weights. Here we dynamize on aggregate values.

2. An $O(n)$-time exact algorithm for FKP based on a greedy method.
3. An $O(n)$-time 2-approximation algorithm for 01KP based on (2).
4. An $O\left(\frac{n^2}{\varepsilon}\right)$-time FPTAS for 01KP based on value-scaling and (1, 3).

   This result is due to Ibarra-Kim [1975].

   The book by Kellerer-Pferschy-Pisinger [2004] gives improved bounds.

01KP by Dynamic Programming

- $V_{UB} \stackrel{\text{def}}{=}$ an upper bound on the aggregate value of the optimum solution to the 01KP input instance, e.g., $V_{UB} = \sum_{i=1}^{n} v_i$

- For $j = 0\,..\,n$, $V = 0\,..\,V_{UB}$ define:

  - $S_{opt}(j,V) \stackrel{\text{def}}{=}$ the subset $S \subseteq \{1, \ldots, j\}$ with minimum aggregate weight subject to: $\sum_{i \in S} w_i \leq W$ and $\sum_{i \in S} v_i = V$

  - $MinW(j,V) \stackrel{\text{def}}{=} \sum_{i \in S_{opt}(j,V)} w_i$ aggregate weight of $S_{opt}(j,V)$
    ($MinW(j,V) = \infty$ if $S_{opt}(j,V)$ does not exist)

  - $x(j,V) \stackrel{\text{def}}{=} \begin{cases} 1 & if\ item\ j \in S_{opt}(j,V) \\ 0 & otherwise \end{cases}$

DP recurrences

$$MinW(j,V) = \begin{cases} 0 & if\ j = 0, \quad V = 0 \\ \infty & if\ j = 0, \quad V \in 1..V_{UB} \\ MinW(j-1, V-v_j) + w_j & \begin{cases} if\ j \geq 1,\ v_j \leq V \quad and \\ MinW(j-1, V-v_j) + w_j \\ \leq min\{W, MinW(j-1,V)\} \end{cases} \\ MinW(j-1,V) & otherwise \end{cases}$$

$$x(j,V) = \begin{cases} 1 & if\ j \geq 1, v_j \leq V, MinW(j-1, V-v_j) + w_j \leq min\{W, MinW(j-1,V)\} \\ 0 & otherwise \end{cases}$$

01KP exact algorithm in $O(nV_{UB})$ time

ALGORITHM1: 01KP Dynamic Programming (v, w, W, V_{UB})

$MinW(0,0) \leftarrow 0$; **for** $j \leftarrow 1..V_{UB}$ **do** $MinW(0,V) \leftarrow \infty$

for $j \leftarrow 1..n$ **do**

  for $V \leftarrow 0..V_{UB}$ **do** $x(j,V) \leftarrow 0$; $MinW(j,V) \leftarrow MinW(j-1,V)$

  for $V \leftarrow v_j..V_{UB}$ **do**

    if $MinW(j-1, V-v_j) + w_j \leq \min\{W, MinW(j,V)\}$ **then**

      $x(j,V) \leftarrow 1$; $MinW(j,V) \leftarrow MinW(j-1, V-v_j) + w_j$

$S_{OPT} \leftarrow \emptyset$; $\bar{V} \leftarrow \max\{V \in 0..V_{UB} \mid MinW(n,V) < \infty\}$

for $j \leftarrow n$ *downto* 1 **do**

  if $x(j,\bar{V}) = 1$ **then** $S_{OPT} \leftarrow S_{OPT} \cup \{j\}$; $\bar{V} \leftarrow \bar{V} - v_j$

return S_{OPT}

Fractional-KP exact algorithm in $O(n)$ time

ALGORITHM2: FKP Greedy (v, w, W)

1. consider items in decreasing order of v_j / w_j :

$$\frac{v_1}{w_1} \geq \frac{v_2}{w_2} \geq \cdots \geq \frac{v_{k-1}}{w_{k-1}} \geq \frac{v_k}{w_k} \geq \cdots \geq \frac{v_n}{w_n}$$

2. place the items in the knapsack in that order until it fills up:

$$k \leftarrow \min\left\{ j \in \{1..n\} \mid \sum_{i=1}^{j} w_i > W \right\}$$

$$x_j \leftarrow \begin{cases} 1 & j = 1..k-1 \\ \left(W - \sum_{i=1}^{k-1} w_i\right)/w_k & j = k \\ 0 & j = k+1..n \end{cases}$$

3. **return** x

Time Complexity:

Only the last item k placed in the knapsack may be fractionally selected.
Steps 1-2 can be done by sorting in O(n log n) time.
Instead, we can use weighted selection in O(n) time.
(See [CLRS] Problem 9-2, or my EECS 3101 LS5.)

Exercise: Show this greedy strategy is correct for FKP, but fails to find the exact 01KP solution (when the fractional item is discarded).

01KP 2-approx algorithm in $O(n)$ time

ALGORITHM3: 01KP 2-approximation (v, w, W)

1. consider items in decreasing order of v_j / w_j :

$$\frac{v_1}{w_1} \geq \frac{v_2}{w_2} \geq \cdots \geq \frac{v_{k-1}}{w_{k-1}} \geq \frac{v_k}{w_k} \geq \cdots \geq \frac{v_n}{w_n}$$

2. $k \leftarrow \min\left\{ j \in \{1..n\} \mid \sum_{i=1}^{j} w_i > W \right\}$

3. $S_1 \leftarrow \{1..k-1\}; \quad S_2 \leftarrow \{k\}$ (∗ both are 01KP feasible solutions ∗)

4. **if** $V(S_1) > V(S_2)$ **then** $S^* \leftarrow S_1$ **else** $S^* \leftarrow S_2$
5. **return** S^*

2-approximation:

$$2V(S^*) \geq V(S_1 \cup S_2) = V(\{1..k\}) \geq FKP\, V_{OPT} \geq 01KP\, V_{OPT}.$$

01KP approximation by scaling values

- Consider the $O(nV_{UB})$ time DP exact algorithm for 01KP
- Scale (and round) down the item values by some factor $s \geq 1$
- Don't alter weights or knapsack capacity
- This does not affect the set of feasible solutions to 01KP
- Running time is scaled down to $O(nV_{UB}/s)$
- How much is the value of each feasible solution scaled down?
- The optimum subset of items may not be optimum in the scaled version!
- What is the relative error?

- **Example:**

  v_1 = 327,901,682 , v_2 = 605,248,517 , v_3 = 451,773,005

  scaling factor: s = 1000,000

  scaled values: $\bar{v}_1$ = 327 , $\bar{v}_2$ = 605 , $\bar{v}_3$ = 451

FPTAS for 01KP by scaling values

ALGORITHM4: $01\text{KP}-FPTAS(v[1..n], w[1..n], W, \varepsilon)$

1. $S_1 \leftarrow$ 2-approximate solution returned by greedy **Algorithm3** (v, w, W)
2. $s \leftarrow \max\left\{1\,,\ \frac{\varepsilon V(S_1)}{n}\right\}$; $V_{UB} \leftarrow \left\lfloor \frac{2V(S_1)}{s} \right\rfloor$ (* scaled upper bound *)
3. **for** $j \leftarrow 1..n$ **do** $\overline{v}_j \leftarrow \left\lfloor \frac{v_j}{s} \right\rfloor$ (* scaled item values *)
4. $S_2 \leftarrow$ solution returned by DP **Algorithm1** $(\overline{v}, w, W, V_{UB})$
5. **if** $V(S_1) > V(S_2)$ **then** $S^* \leftarrow S_1$ **else** $S^* \leftarrow S_2$
6. **return** S^*

Time Complexity: $O(n + nV_{UB}) = O\left(\frac{2nV(S_1)}{s}\right) = O(n^2/\varepsilon)$.

FPTAS for 01KP by scaling values

ALGORITHM4: $01\text{KP}-FPTAS(v[1..n], w[1..n], W, \varepsilon)$

1. $S_1 \leftarrow$ 2-approximate solution returned by greedy **Algorithm3** (v, w, W)
2. $s \leftarrow \max\left\{1\,,\ \frac{\varepsilon V(S_1)}{n}\right\}$; $V_{UB} \leftarrow \left\lfloor \frac{2V(S_1)}{s} \right\rfloor$ (* scaled upper bound *)
3. **for** $j \leftarrow 1..n$ **do** $\overline{v}_j \leftarrow \left\lfloor \frac{v_j}{s} \right\rfloor$ (* scaled item values *)
4. $S_2 \leftarrow$ solution returned by DP **Algorithm1** $(\overline{v}, w, W, V_{UB})$
5. **if** $V(S_1) > V(S_2)$ **then** $S^* \leftarrow S_1$ **else** $S^* \leftarrow S_2$
6. **return** S^*

ε relative error: $s = 1 \implies V(S^*) = V(S_2) = V(S_{OPT})$.

$$s = \frac{\varepsilon V(S_1)}{n} \implies V(S_2) = \sum_{j\in S_2} v_j \geq \sum_{j\in S_2} s\,\overline{v}_j = s\sum_{j\in S_2} \overline{v}_j \geq s\sum_{j\in S_{OPT}} \overline{v}_j$$

$$= \sum_{j\in S_{OPT}} s\,\overline{v}_j \geq \sum_{j\in S_{OPT}} (v_j - s) \geq V(S_{OPT}) - ns = V(S_{OPT}) - \varepsilon V(S_1)$$

$$\implies \quad V(S_{OPT}) \ \leq\ V(S_2) + \varepsilon V(S_1) \ \leq\ (1+\varepsilon)\,V(S^*)$$

$$\implies \quad V(S^*) \ \geq\ (1-\varepsilon)\,V(S_{OPT})\,.$$

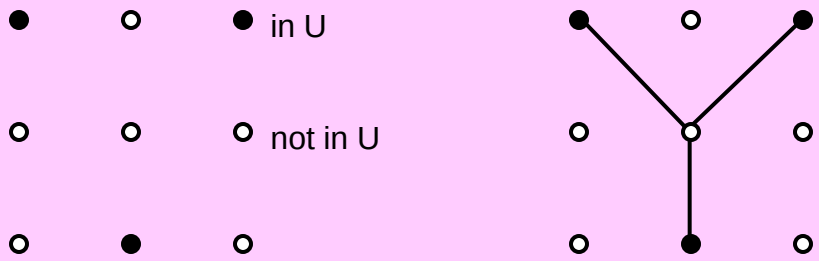
Exercises

1. **[CLRS, Exercise 35.1-4, page 1111] Vertex Cover of a tree:** Consider the special case of the vertex cover problem when the given graph is a tree (an acyclic connected graph). The question is, is this special case still NP-hard, or is it solvable **exactly** (not approximately) in polynomial time?
   (a) Show that the minimum **un-weighted** vertex cover of a tree can be computed in linear time.
   (b) Can the minimum **weighted** vertex cover of a tree be computed in polynomial time?
   If yes, describe one such efficient algorithm.

2. **Maximum degree greedy heuristic for un-weighted Vertex Cover Problem:**
   Consider the following greedy strategy: select a vertex of maximum degree, place it in the cover, and remove it and all its incident edges. Repeat on this greedy iteration until there are no more edges.
   Give an example to show that the approximation ratio for this heuristic is more than 2.
   [Hint: Try a bipartite graph with vertices of uniform degree on the left and varying degree on the right.]

3. **Greedy heuristic for weighted Vertex Cover Problem:**
   Modify the greedy selection criterion of the heuristic in the previous exercise by repeatedly selecting a vertex v with minimum *w(v)/degree(v)*. What can you conclude? Is there a connection between this strategy and the pricing method for the Weighted Set Cover Problem?

4. **The Metric Minimum Steiner Tree Problem:** The input to the *Minimum Steiner Tree Problem (MSTP)* is a complete graph $G(V, E)$ with distances d_{uv} defined on every pair (u, v) of nodes; and a distinguished set of *terminal nodes* $U \subseteq V$. The goal is to find a minimum-cost tree that includes the terminal nodes U. This tree may or may not include nodes in $V - U$.
   In Metric-MSTP the distances in the input form a *metric*. This is known to be an NP-hard problem.
   Show that an efficient 2-approximation algorithm for Metric-MSTP can be obtained by ignoring the non-terminal nodes and simply returning the minimum spanning tree on U.



5. E-Commerce Advertising:

You are consulting for an e-commerce site that receives a large number of visitors each day. You are facing the off-line problem described below. For each visitor i, where $i \in \{1, 2, \ldots, n\}$, the site has assigned a value v_i, representing the expected revenue that can be obtained from this customer. Each visitor i is shown one of m possible ads $A_1, A_2, \ldots, A_m$ as they enter the site. The site wants a selection of one ad for each customer so that *each* ad is seen, overall, by a set of customers of reasonably large total value. Thus, given a selection of one ad for each customer, we will define the *spread* of this selection to be the minimum, over $j = 1, 2, \ldots, m$, of the total value of all customers who were shown ad A_j.

Example: Suppose there are six customers with values 3, 4, 12, 2, 4, 6, and there are $m = 3$ ads. Then, in this instance, one could achieve a spread of 9 by showing ad A_1 to customers 1, 2, 4, ad A_2 to customer 3, and ad A_3 to customers 5 and 6.

The ultimate goal is to find a selection of an ad for each customer that maximizes the spread. Unfortunately, this optimization problem is NP-hard (you don't have to prove this). So, instead, we will try to approximate it.

(a) Design and analyze a polynomial-time 2-approximation algorithm for the maximum spread problem. [That is, if the maximum spread is s, then your algorithm should produce a selection of one ad per customer that has spread at least $s/2$.]

(b) Give an example of an instance on which the algorithm you designed in part (a) does not find an optimal solution (i.e., one of maximum spread). Say what the optimal solution is in your sample instance, and what your algorithm finds.

6. The K Emergency Location Problem:

Consider a weighted complete undirected graph G on a set V of n vertices. Let d_{ij} denote the nonnegative weight (distance) of edge (i, j) in G. We want to select a subset S of the vertices, with $|S| = K$, so that the longest distance of any vertex of G from its nearest vertex in S is minimized. (You can imagine we want to set up K emergency locations in the city, such as hospitals or fire stations, so that no location in the city is too far from its nearest emergency location.)

Formally, the objective is to find $S \subseteq V$, $|S| = K$, that minimizes the quantity

$$cost(S) = \max_{i \in V} \min_{j \in S} d_{ij} .$$

The *metric* version of the problem is one in which the distances satisfy the metric axioms. Even the metric version of the problem is NP-hard. Consider the following greedy heuristic:

$S \leftarrow \varnothing$
Select a vertex $v \in V$ arbitrarily
loop K times
  $S \leftarrow S \cup \{v\}$
  $V \leftarrow V - \{v\}$
  Select $v \in V$ so that its closest site in S is as far as possible, i.e.,
    $\min_{j \in S} d_{vj} = \max_{i \in V} \min_{j \in S} d_{ij}$.
end loop
return S

(a) Prove that this greedy heuristic is a 2-approximation algorithm for the Metric K Emergency Location Problem.

(b) Develop an implementation of this algorithm that runs in O(nK) time.

7. The Metric K-Supplier Problem:

We are given an edge-weighted complete graph G(V, E). Assume the edge weights d_{ij}, for all (i, j)∈E, form a metric. We are also given a partitioning of the vertex set V into a non-empty proper subset S of *suppliers* and a set C = V – S of *customers*, and a positive integer K.

For any subset A ⊆ S of suppliers and any customer $c \in C$, we define the distance of c from A, denoted $d(c, A)$, as the distance from c to its nearest neighbor in A, i.e.,

$$d(c, A) = min \{ d_{cs} \mid s \in A \}.$$ [Note: $d(c, \emptyset) = \infty$.]

The problem is to select a subset A ⊆ S , such that $|A| \le K$, to minimize the objective function

$$f(A) = max \{ d(c, A) \mid c \in C \}$$

That is, we want to minimize the largest distance of any customer to its nearest supplier in A. This problem is NP-hard. Consider the following greedy approximation algorithm:

$A \leftarrow \emptyset$
for i ← 1 .. K **do**
  select $c \in C$ farthest from A (i.e., $d(c, A) = f(A)$)
  select $s \in S$ closest to c (i.e., $d_{cs} = d(c, S)$)
  $A \leftarrow A \cup \{s\}$
end for
return A

(a) Prove that this is a 3-approximation algorithm. [Hint: apply the pigeon-hole principle.]

(b) Show the approximation ratio ρ=3 is tight for this algorithm. [Hint: show an instance with Euclidean metric and K =1 for which ρ is (arbitrarily close to) 3.]

8. **Closest Point Insertion Heuristic for metric-TSP**:
Begin with the trivial cycle consisting of a single arbitrarily chosen vertex. At each step identify the vertex *u* that is not on the cycle but whose distance to the closest vertex on the cycle is minimum possible. Suppose that the vertex on the cycle nearest *u* is *v*. Extend the cycle to include *u* by inserting *u* just after *v* on the cycle. Repeat until all vertices are on the cycle. Prove that this heuristic is a 2-approximation algorithm for the metric-TSP. Also show how to implement the algorithm to run in $O(n^2)$ time, where *n* is the number of vertices.

9. **The Bottleneck metric-TSP:**
The problem is to find a TSP cycle such that the cost of the most costly edge on the cycle is minimized. Assume edge lengths form a metric. Show a polynomial-time 3-approximation algorithm for this problem. [Hint: Show recursively that we can visit all the nodes in a bottleneck spanning tree, as discussed in [CLRS: Problem 23-3], exactly once by taking a full walk of the tree and skipping nodes, but without skipping more that two consecutive intermediate nodes. Show that the costliest edge in the bottleneck spanning tree has a cost that is at most the cost of the costliest edge in a bottleneck TSP tour.]

END